Republic of Tunisia
MINISTRY OF HIGHER EDUCATION AND SCIENTIFIC RESEARCH
University Of Tunis El-Manar
National Engineering School Of Tunis

Information and Communication Technologies Department

Graduation Project

Elaborated by

Elyess Ben Amor

To Obtain

The National Telecommunication Engineering Degree

CI/CD Pipelines For Migrating & Monitoring VARAO Platform To Private Cloud

Realized In

vorwärts
the amazon experts

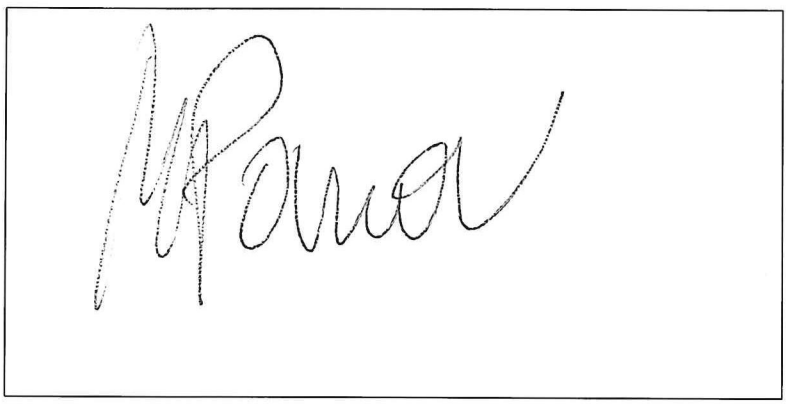
Presented on 31/10/2020

Jury members:

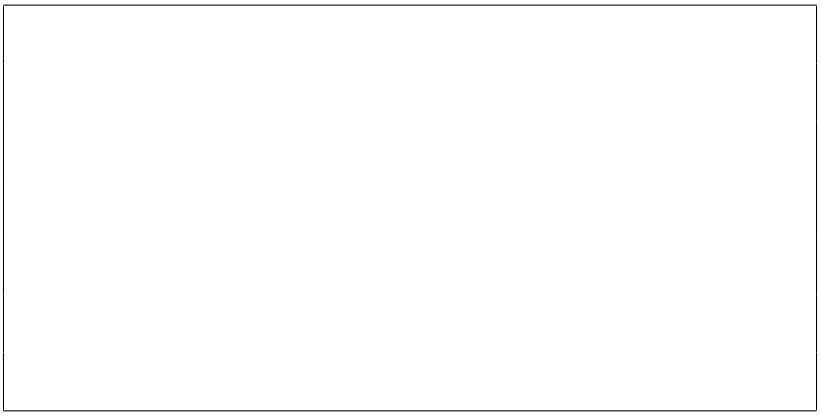
President	: Mr.Mohamed ESCHEIKH
Examiner	: Mrs.Soumaya MEHERZI
Company supervisor	: Mr.Tobias Gurtzick and Mr.Mladen Panov
Academic supervisor	: Mrs.Mariem Kassar

Academic year 2019/2020

Signatures



Signatures



Acknowledgements

I would like to extend my sincere thanks to all the team members of Vorwaerts company for their support in my graduation project.

I am greatly indebted to **Mr.Tobias Gurtzick** and **Mr.Mladen Panov** for their constant supervision and guidance as well as for providing me with the necessary information in regards to the project and also for his continuous support in completing the graduation project.

I would also like to extend my deepest gratitude to **Ms.Meriam Kassar** for her kind cooperation . My appreciation go to my friends and people who have willingly helped me out with their abilities and a big thank to Tunisian Cloud Native members.

abstract

In the last decade, the software industry has been marked by a growing trend of software companies' ability to deploy new software features fast and frequently in short release cycle times. The companies' software release cycles have been shortened to hours and minutes rather than months.

To enable the transformation towards short release cycle times, companies have adopted several different strategies, including the DevOps approach. DevOps in the software industry emerged to represent a professional movement emphasising the collaboration between software development and operations. In practice, DevOps affects the company culture, processes, products, associated technologies and organisational structures used in software development and operations processes.

The purpose of this final project is the integration of DevOps culture in the company and the CI/CD approach in an attempt to remove the gap between the Devs and Ops teams ard facilitate the deployment chain by automating the maximum of tasks.

Key words: DevOps, Automation, Cloud, CI/CD, Docker, Kubernetes, Gitlab, Rancher, Nodejs , Prometheus , OpenEBS/NFS server etc.

Contents

List of Figures **8**

List of tables **11**

List of acronyms **12**

General Introduction **14**

1 Project Presentation **17**

Introduction 17

1.1 Company Presentation 17

1.2 Organization chart 18

1.3 Project Context 18

1.3.1 Scope 18

1.3.2 Project Challenges 18

1.3.3 Project Objectives 19

1.4 Services And Tools 20

1.4.1 Amazon SEO 20

1.4.2 SaaS Applications 20

1.4.2.1 VAPA: AI-Driven Autonomous Bid Management 20

1.4.2.2 VARAO 21
1.4.2.3 CDP 22
1.5 Work methodology 22
1.5.1 Method Adopted 23
1.5.2 SCRUM Methodology 23
1.5.3 Operation principles 24
Conclusion 25

2 DevOps Background 26

Introduction 26
2.1 DevOps Definition 26
2.2 The traditional life cycle of a software 27
2.3 DevOps life cycle 27
2.3.1 Continuous Development 28
2.3.2 Continuous Integration 29
2.3.3 Continuous Deployment 30
2.3.4 Continuous Monitoring 30
2.3.5 Continuous Testing 31
2.4 Key elements of DevOps 31
Conclusion 32

3 State Of Art 33

Introduction 33
3.1 Cloud Design 33
3.1.1 Study Of The Existing 33

3.1.1.1 Amazon Web Service 33
3.1.1.2 Microsoft Azure 34
3.1.1.3 Heroku 35
3.1.1.4 IBM Cloud 35
3.1.1.5 Google Cloud Platform 36
3.1.2 Critics 36
3.1.3 Proposed Solution 36
3.2 Cloud Architecture 37
3.2.1 Hardware Component 37
3.2.2 Storage Component 38
3.3 Network And Virtualisation Architecture 39
3.3.1 Docker 39
3.3.1.1 Basic Concept 39
3.3.1.2 Docker Fundamentals 39
3.3.1.3 Docker Architecture 40
3.3.2 Kubernetes 41
3.3.2.1 Overview 41
3.3.2.2 Kubernetes Components 42
3.3.2.3 Kubernetes Deployment Kinds 45
3.3.3 Rancher 46
Conclusion 47

4 CI/CD pipeline Tools 48

Introduction 48

4.1 Pipeline Architecture 48

4.1.1 Goals 48

4.1.1.1 Technical Goals 48

4.1.1.2 Functional Goals 49

4.1.1.3 Product Backlog 49

4.1.2 DevOps Tools 49

4.1.2.1 GitLab 50

4.1.2.2 Kaniko 52

4.1.2.3 Helm 52

4.1.2.4 Cert-Manager 53

4.1.2.5 OpenEBS 54

4.2 Monitoring Concept 56

4.2.1 Open Source tools 56

4.2.1.1 Prometheus 56

4.2.1.2 EFK Stack 59

4.2.1.3 Bull Exporter 60

4.2.2 Horizontal Pod Autoscaling 60

4.2.2.1 Overview 60

4.2.2.2 HPA Architecture 62

Conclusion 63

5 Release **64**

Introduction 64

5.1 Charts Preparation 64

5.1.1 Overview 64

5.1.2 Charts Components 65

5.1.2.1 Containerisation 65

5.1.2.2 Kubernetes Deployments 65

5.2 Gitlab Pipeline 69

5.2.1 Design 69

5.2.2 Implementation 70

5.2.2.1 Test 72

5.2.2.2 Build 72

5.2.2.3 Deploy 74

5.3 Monitoring Configuration 76

5.3.1 Rancher Embedded Monitoring 76

5.3.2 EFK Container Logging 77

5.3.2.1 EFK Architecture 77

5.3.2.2 EFK Deployments 78

5.3.2.3 Container Tracking 79

5.3.3 Prometheus Configuration 81

5.3.3.1 Integration 81

5.3.3.2 Autoscaling 85

Conclusion 86

General conclusion and Perspectives **86**

Bibliographie **87**

List of Figures

1.1 **VAPA Platform [1]** 21

1.2 **VARAO Platform [2]** 22

1.3 **CDP Platform [3]** 23

1.4 **SCRUM Process [4]** 25

2.1 **DevOps Terms[5]** 27

2.2 **DevOps Life Cycle[6]** 28

2.3 **Development Phase[7]** 29

2.4 **Continuous Integration Pipeline[8]** 29

2.5 **Deployment Phase[9]** 30

2.6 **Testing Phase[10]** 31

3.1 **Docker Architecture[11]** 41

3.2 **Pod Creation and API concept[12]** 43

3.3 **Rancher UI Overview[13]** 47

4.1 **Gitlab Features[14]** 51

4.2 **Simple Gitlab CI/CD[15]** 52

4.3 **Helm Architecture[16]** 53

4.4 **OpenEBS Local PV Provisioning[17]** 55

4.5 **OpenEBS Installation[18]** 56

4.6 **Prometheus Operator Architecture[19]** 58

4.7 **Fluentd Mechanism [20]** 59

4.8 **EFK Architecture [21]** 60

4.9 **HPA Control Loop [22]** 61

4.10 **HPA Architecture [23]** 62

5.1 **VARAO Dockerfile[24]** 65

5.2 **VARAO Yaml File Deployment[25]** 66

5.3 **VARAO Yaml File Service[26]** 67

5.4 **VARAO Yaml File SSL Certification[27]** 68

5.5 **VARAO Yaml File SSL Certification[28]** 68

5.6 **Four Stage Pipeline[29]** 69

5.7 **Gitlab Environment Configuration[30]** 70

5.8 **Gitlab Stages And Modules[31]** 71

5.9 **Gitlab Test Stage[32]** 72

5.10 **Gitlab Build Stage[33]** 73

5.11 **Gitlab Deploy Stage[34]** 75

5.12 **Embedded Rancher Monitoring[35]** 76

5.13 **VARAO Grafana Dashboard [36]** 77

5.14 **EFK Gitlab Project Structure [37]** 78

5.15 **Logtrail Pluging [38]** 78

5.16 **Kibana Dashboard [39]** 80

5.17 **Logtrail Dashboard [40]** 80

5.18 **Operator Project Structure[41]** 81

5.19 **Prometheus Service Monitore File[42]** 82

5.20 **URL Specification[43]** 82

5.21 **Rules Specification[44]** 83

5.22 **Bull Deployment[45]** 84

5.23 **Bull Service[46]** 85

5.24 **Bull Service Monitor[47]** 85

5.25 **HPA File[48]** 86

List of Tables

2.1 Key Elements (characteristics) of DevOps 32

3.1 Features And Limits of AWS 34

3.2 Features And Limits of Azure 34

3.3 Features And Limits of Heroku 35

3.4 Features And Limits of IBM Cloud 35

3.5 Features And Limits of GCP 36

4.1 Product Backlog 49

List of acronyms

API: Application Programming Interface Unit

ASIN: Amazon Standard Identification Number

AWS: Amazon Web Services

CPU: Central Processing

CDP: Club Der Produkttester

CI: Continuous Integration

CD: Continuous Delivery

CNCF: Cloud Native Computing Foundation

CAS: Container Attached Storage

CRD: Custom Resource Definitions

EFK: Elasticsearch FluentBit Kibana

EBS: Elastic Block Storage

GCP: Google Cloud Platform

IaaS: Infrastructure as a service

IT: Information Technology

IBM: lnternational Business Machines

NAS: Network Attached Storage

OVH: Online Virtual Hosting

PaaS: Platform as a Service

RAM: Random Access Memory

RBAC: Role-Based Access Control

SaaS: Software as a Service

TLS: Transport Layer Security

VÀPA: Vorvwaerts Amazon Programmatic Advertising

VARAO: Vorvwaerts Amazon Research And Optimisation

YAML: Yet Another Markup Language

General Introduction

Software has become pervasive in day-to-day human activities and the economy. This in turn has increased the importance of having software-intensive products and services that are useful, secure and reliable at all times during operational use. Recently, the ability of an organisation to continuously and rapidly provide new software features that are suitable to use and innovate has become an important factor of competition in the software industry market.

Continuous deployment entails the capability of an organisation to deliver new software features at multiple times and in the shortest time possible. DevOps is an approach that has been reported to enable the continuous deployment paradigm as it embodies a set of useful principles crucial to the development and deployment of software.

In fact, DevOps is a uses term emerging from the collision of two major related trends. The first was also called ”agile infrastructure” or ”agile operations”; it sprang from applying Agile and Lean approaches to operations work. The second is a much expanded understanding of the value of collaboration between development and operations staff throughout all stages of the development life-cycle then creating and operating a service, and how important operations has become in our increasingly service-oriented world.

This project contains eight chapters, whose contents are logically structured as follows.
The first chapter presents the hosting company Vorwaerts GmbH, its organizational charts, services and tools that uses for supporting multiple clients and the project objectives.
The second chapter presents the background and existing literature of the DevOps concept, its key characterising elements, practices, benefits and challenges.

In Chapter three, we present the research methods used to consolidate the knowledge of the adoption of DevOps in practice are presented.we will walk through the methodology adopted and define how roles are assigned.

In the fourth chapter we are going to talk about the work methodology and the general scope of the project in which we find the study of the challenges and how to adopt the best solution.

In the fifth chapter, we will present the private cloud architecture and detail its components and technology stack used, then we will present the services offered through it.Then we are going to make some critics about the used architecture.

The sixth chapter is dedicated to the main product we are going to work on which is CI/CD pipelines, where we will define the actors and requirements of this product, then we will present a description of the pipeline main parts, the technology stack we will use the build it and our work environment.

Last but not least we will illustrate the achieved work and how it was implemented in order to evaluate the work.

Finally we will see what must be done to add some value to our work and on how to monitor applications with proposed solution before getting to the general conclusion in which we will outline future perspectives.

Chapter 1

Project Presentation

Introduction

My final project internship took place within the Vorwaerts company. In this part, we will introduce the host company and its different services related to its staff organization and work methodology. Then, we are going to present the main challenges and objective of our project.

1.1 Company Presentation

Vorwârts GmbH is an international Amazon marketing company created in 2007 and is based in Germany with Sates representatives in United Kingdom. Vorwârts GmbH has be- come a leading marketing company for different Amazon online markets across Europe. [1] Vorwärts GmbH specializes in online business models, online marketing and SaaS applica- tions for automating Amazon SEO. Its customers and partners include well known brands and groups from the automotive, medical. tourism, postal services, security and online services sectors.

1.2 Organization chart

Vonvârts GmbH's team is composed of IT engineers, Data scientists, Amazon managers. Business Analysts, Project and marketing managers and C-level officers, The team members have different,yet. complementary skills. Some of these disciplines are automation, software development, quality management and marketing. During the internship. wee integrated the developers. DevOps, testers and marketing team under the supervision of a lead development engineer.

1.3 Project Context

1.3.1 Scope

This project is in the context of a graduation from the **National Engineering School Of Tunis**. It was made during an internship in Germany and took place at Vorwaerts GmbH. It consist in general in containerizing VARAO plateform, deploy it using orchestration plateform and ensure monitoring during the CI/CD pipeline.

1.3.2 Project Challenges

Cloud computing technology is being widely used by business companies around the globe to manages their application and big data, however there is a gap between success and failure in businesses.
By choosing the right cloud solution, it might take the company to new heights and using today's DevOps trends we could say that success will be related to teams cooperation while deploying applications.
Yet, using cloud solution is not sufficient because application are growing in term of packages, volumes in servers, user requests and many other needs so that is why containers have become the next big thing in infrastructure software.However, in order to take full advantage of containers we need to know how to turn them into production services.
In this context comes the decision of containerizing VARAO platform and deploy it into kubernetes while managing certain cloud constraint present in these following challenges:

- **High Availability**: Treating each instance of software as a disposable entity without any degrade in its functionalities.
- **Performance and Scalability**: Ensure performance of the different application copies by optimizing its access to databases.
- **Security**: Manage security threats such us virus and hacking by applying rules like security policies and network policies.
- **Monitoring**: Ensure debugging distributed services running in containers across the network boundaries will help IT developer to know better how the application will react.

1.3.3 Project Objectives

As a part of its policy that aims to follow technological progress and to improve its services, our project's main objective is to implement VARAO platform as microservice application, set up the whole pipeline, use monitoring tools to detects application metrics for better auto-scaling and deploy it to our private cloud in a way that fits with the vision state of many challenges. Therefore, we set our objectives to:

1. Dockerize VARAO application.
2. Create yaml file charts of VARAO platform to deploy it over kubernetes.
3. Implement CI/CD pipelines for VARAO platform's different services.
4. Use open-source monitoring tools and integrate them with the architecture of VARAO application.
5. Ensure the monitoring of the platform.
6. Ensure HPA for VARAO plateform.
7. Automate ip failover of OVH cloud provider.

1.4 Services And Tools

As a leader in the International market in marketing and selling amazon products , Vorwaerts company uses different tools and services that allow her to be more efficient in IT and marketing stuff.

1.4.1 Amazon SEO

The core objective of this service is to increase products rank to appear at the top of the Amazon search and to empower the sales of the products. For each product, Vorwârts analyses the competition, the target groups, device searches from behavior patterns, build a relevance based on these for the products, and ensure that the performance of the product listings, after the optimization in terms of click through rate and conversion rate, reaches the maximum possible values Sales Rank is maximally positively influenced. [2]

1.4.2 SaaS Applications

This product category includes three SaaS products. The products are VARAO. VAPA and CDP.

1.4.2.1 VAPA: AI-Driven Autonomous Bid Management

The goal of Amazon marketing automation is to ensure you stay on target to reach your marketing goals and achieve the highest Amazon turnover from every marketing Euro spent.
The solution for this is VAPA, the AI-driven, fully automated and self-learning advertising tool for the Amazon Advertising Console. You only need to set key parameters such as your available media budget, target and break-even spending and your strategy for a product or product portfolio. VAPA then fully autonomously validates and adjusts your marketing campaigns 24/7 for the target outcome, maximising the return on invested media budget while minimising the time required by you to manage your campaigns.
VAPA stores valuable historical data and experience in real-time so it can continue to use it to identify the most efficient running times for each campaign and each ASIN. The

concept of goal-based programmatic campaign groups allows for an in-depth sales analysis at the keyword level. Structure and evaluate your Amazon campaigns by also using your own personal tagging system.[2]

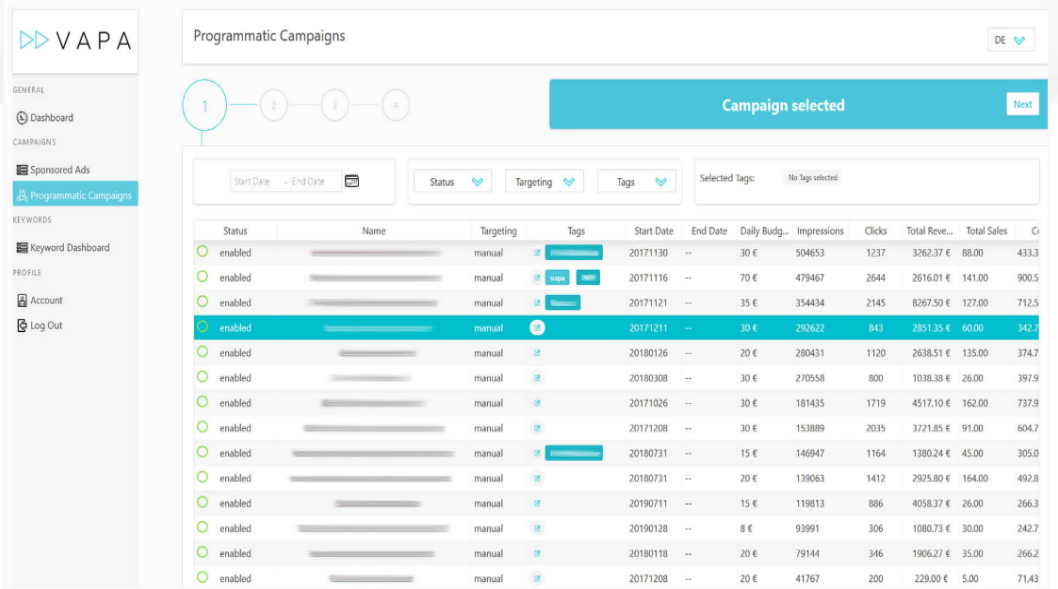


Figure 1.1: **VAPA Platform [1]**

1.4.2.2 VARAO

Without the right keywords, a product will not be listed on the corresponding Amazon search pages nor seen by your customers.

VARAO uses the search results from Amazon and other e-commerce platforms to find the most relevant keywords for your product. Research and generate highly relevant and product-specific keywords to help you create the best possible content for both, the Amazon algorithm and your customer.

VARAO also acts as a central location for your existing optimisations, allowing you to easily manage all your ASINs. Those with a large number of ASINs in their Amazon portfolio are well aware of the importance of keeping track of content and the identified list of relevant search terms. VARAO safely stores all of these in one place for easy and efficient updates and monitoring.[3]

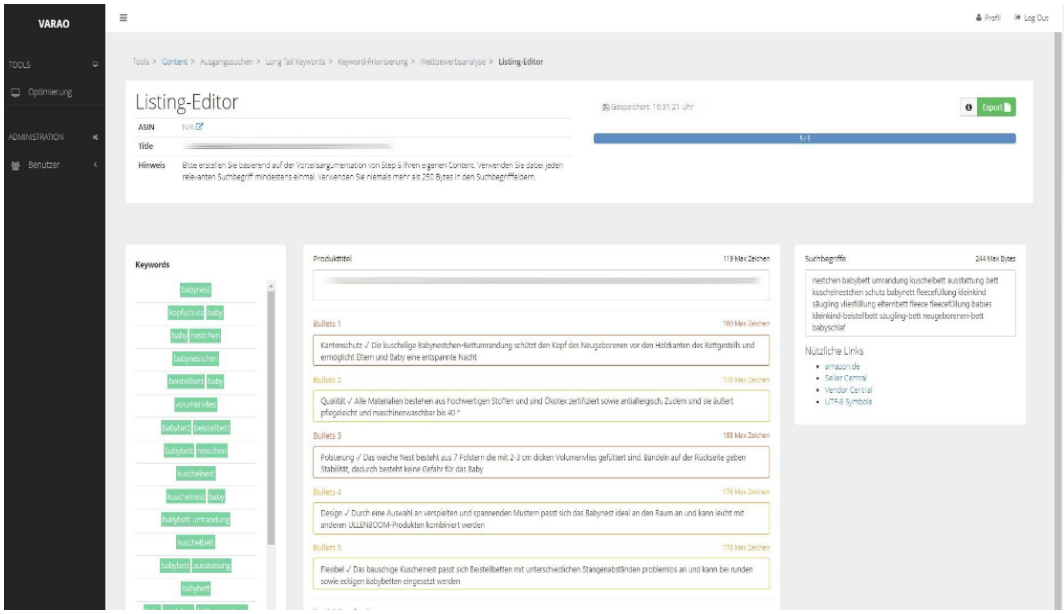


Figure 1.2: **VARAO Platform [2]**

1.4.2.3 CDP

CDP tool is meant to generate real product reviews from independent product testers with particular ease. After booking a membership, you can independently create product tests for any ASIN directly in your login area. You decide how many products (between 3 and 15 per test) you want to have reviewed at the same time.[4]

1.5 Work methodology

The most important key of success is knowing how to maintain the best process chain of a company, so by knowing the methodology adopted, teams will be more aware on how to boost product quality without conflicts. In this section we will present the adopted methodology of the vorwaerts company and its principles.

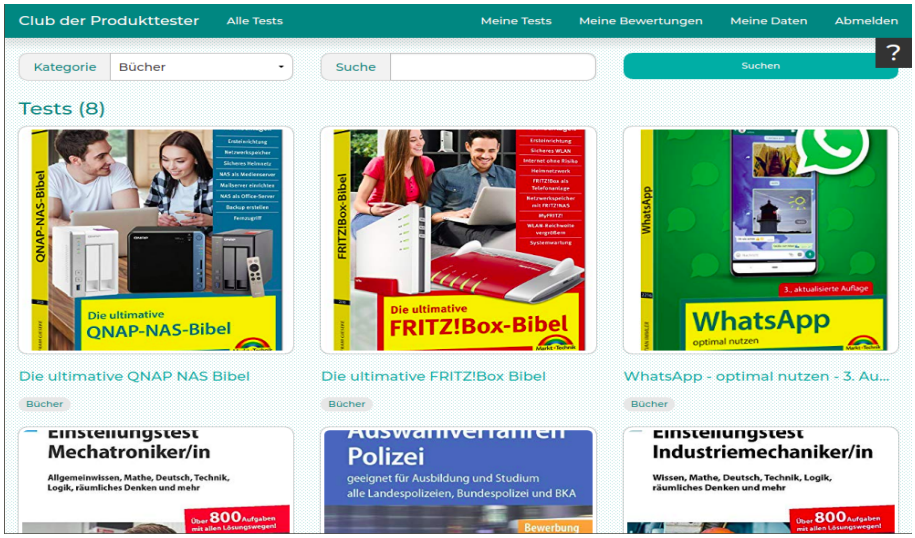


Figure 1.3: **CDP Platform** [3]

1.5.1 Method Adopted

Since the rise of the company , our CTO André adopted SCRUM methodology for all the projects and he assigned for each project a leader. For my case the work team as below:

- **Product Owner :** Hanna Schûtt
- **Scrum Master :** Jasmeen Kosmalla
- **Dev Team :**
  1. Devops : Elyess Ben Amor
  2. Full Stack Developper : Hassen Ben Slama

1.5.2 SCRUM Methodology

SCRUM is a framework that helps team to work together in a consistent way by organizing projects process management.Although this framework is essentially used by software developer but it seems to feet well to any team that is working toward a common goal. As another way to define SCRUM methodology we can say that it is a solution provided for teams to think creatively as they prioritize steps by organizing features and goals into deliverables that the team works on in two-week sprints.

To work with teams in this methodology, there are some roles that must be assigned in order to have a smooth process.

- **scrum product owner :** His role is to understand businesses, customers needs and projects requirements. The product owner is also responsible for prioritizing goals for each sprint ,such us the most important and deliverable features to be built in each iteration.
- **scrum master :** The SCRUM master is responsible for coaching teams and facilitator of the scrum development process. Also he is responsible of the coordination between teams and the product owner.
- **scrum development team :** Those are the individuals who have all the technical,analytical and development skills to carry out the actual work, solve problems and produce deliverable products.

1.5.3 Operation principles

To begin a SCRUM project, teams must respect the process done by the product owner who creates items called product backlog and use feedback from developers/users in order to keep the list of items clean and ready to be worked on [7].
The first step though is **Sprint Planning**, where SCRUM product owner will be fixing the work that must be done in each sprint meeting.This meeting is led by SCRUM master and it is where the team decides on the sprint goal.
Before describing the coming steps we must know what is actually a **Sprint**. A sprint is a period of time when the scrum team work together to synchronize their work and finish an increment.
Another meeting known as **Sprint Review** and it is held at the end of a sprint. In this period of time teams gets in touch with each other to review the increments, also the developers show to the product owner developed features so he can know which are the components that must be released.
The final component of the whole operation is **Daily Scrum Meeting**, this is the shortest meeting that can be done every day at lasts 15 minutes tops allowing everyone to determine

he has achieved since the last meeting.

This figure 1.4 summarize the whole process.

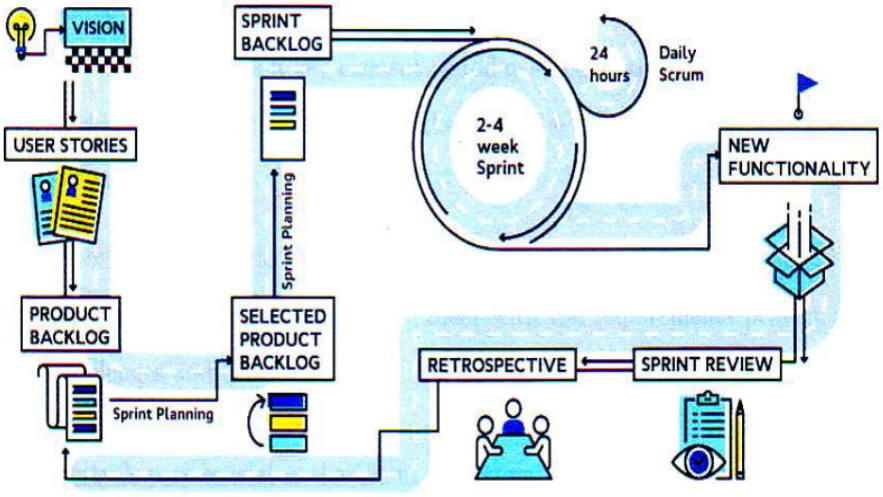


Figure 1.4: **SCRUM Process** [4]

Conclusion

In this chapter, we presented the host company followed by the general context of this project. In order to follow a logical progression in this report. a theoretical study of the methodology applied will be the subject of the next chapter.

Chapter 2

DevOps Background

Introduction

The purpose of this chapter is to introduce the concept of DevOps as well as its adoption, implementation, benefits and challenges in software-intensive organisations based on existing literature.

2.1 DevOps Definition

There is no consensus among software practitioners and scholars of what the DevOps concept entails [8].
In the literature, several definitions of DevOps have been presented. The majority of the descriptions specify DevOps as a term that is used to emphasise the collaboration between software development and operations. The description of software development and operations used in this thesis and presented in Figure 2.1, is derived from ISO/IEC standards and empirical studies describing software development and operations processes.

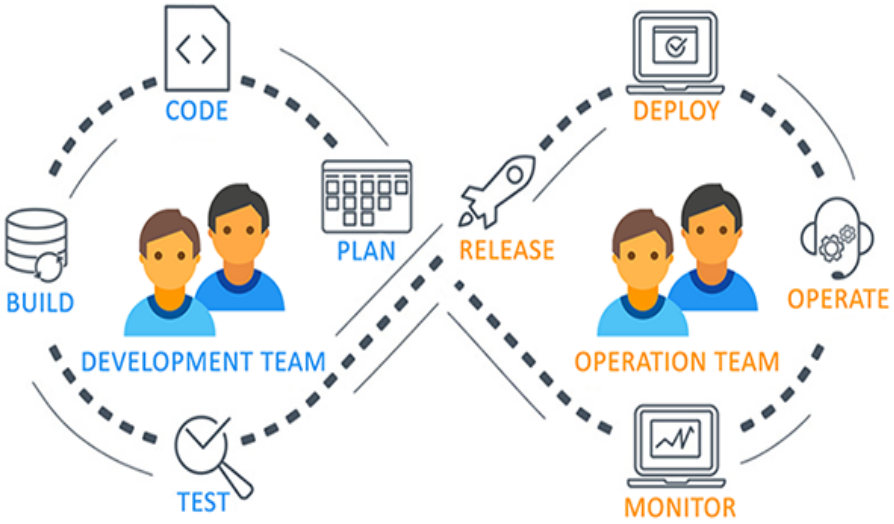


Figure 2.1: **DevOps Terms**[5]

2.2 The traditional life cycle of a software

The traditional life cycle of a software is based on a sequential series of steps like requirements definition, solution building, testing and deployment. Traditional software development methodologies require defining and documenting a stable set of requirements at the beginning of a project [8].

The traditional software development methods are dependent on a set of predetermined processes and on-going documentation which is written as the work progresses and guides further development. The success of a project which is approached in this way relies on knowing all of the requirements before development begin and means that implementing change during the development life cycle can be somewhat problematic. However, it also makes it easier to determine the costs of the project, set a schedule and allocate resources accordingly.[6]

2.3 DevOps life cycle

As mentioned before, DevOps term relies on two parts, the first one is development phase and the second is operating the codes specified by development team.

To see better how it works behind the scenes we must understand deeper the communication between those parts.

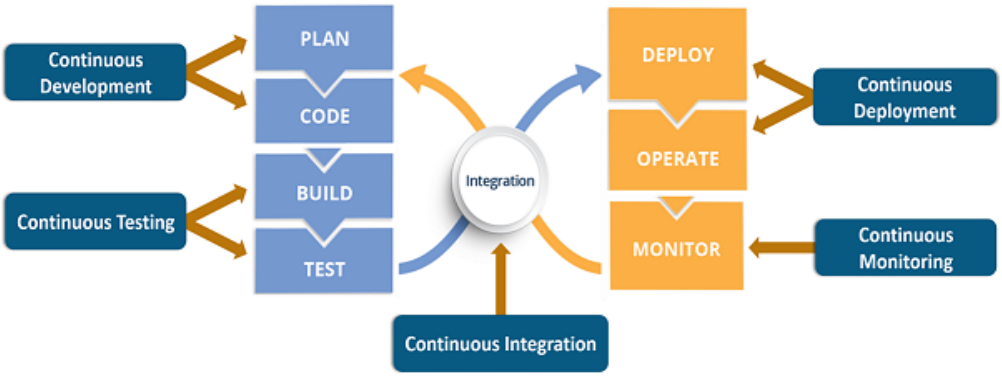


Figure 2.2: **DevOps Life Cycle[6]**

The figure 2.2 shows that DevOps has different ways on how to maintain the life of a product/software or even a plan, For that reason we will be looking at various phases of DevOps life cycle one by one.

2.3.1 Continuous Development

In this part we can find planning and execution of the building codes that means that the vision of the project begin in this phase and the developers begin developing the code for the application.

We must be aware that there are no DevOps tools in this part but we can find tools for maintaining the code and this is the first rule in this phase, so DevOps proposes a concept called Infrastructure as a code (IAAC) which says that complete infrastructure provision should be maintained in source code repository as shown in figure 2.3.

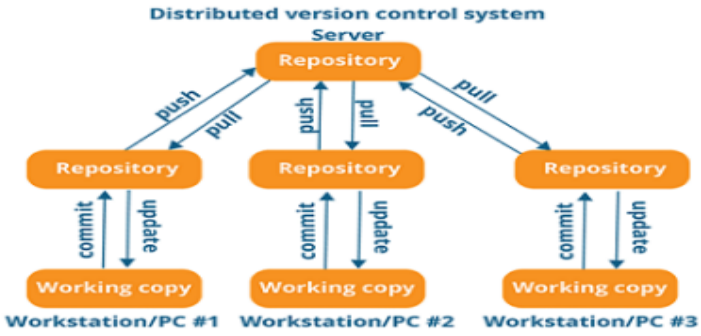


Figure 2.3: **Development Phase**[7]

2.3.2 Continuous Integration

Continuous Integration is a software development where the whole teams integrate their work usually. The result of integrations are verified by automated builds in order to see bugs or errors as quickly as possible.This approach solves many problems and allows a team to develop cohesive software more rapidly.Automating everything in this step is mandatory as shown if figure2.4 [8].

We can mention many benefits of this phase like reducing integration risk, higher code quality, less time deploying ,etc .

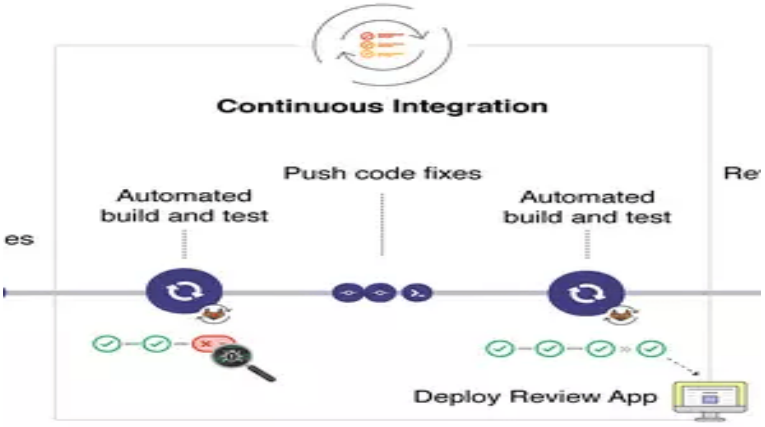


Figure 2.4: **Continuous Integration Pipeline**[8]

2.3.3 Continuous Deployment

Continuous Deployment is main component of the DevOps to the complete software delivery optimization [8]. In this stage all the code is deployed to the production servers by ensuring that the deployments are correctly managed and configured, and we must not forgot that to help raising the production benefits, most of companies use management and containerization tools to achieve CD part.

Configuration Management is maintaining consistency in an application's functional requirements and performance. To define it simpler it is an act of releasing deployments to servers, scheduling updates and maintain the configuration correctly as possible.

Containerization tools helps maintaining application consistency across all environment, In fact most of companies use those tools to add stability, scalability and continuous automation by replicating the same dependencies and packages used in the development/ testing/ staging environment.

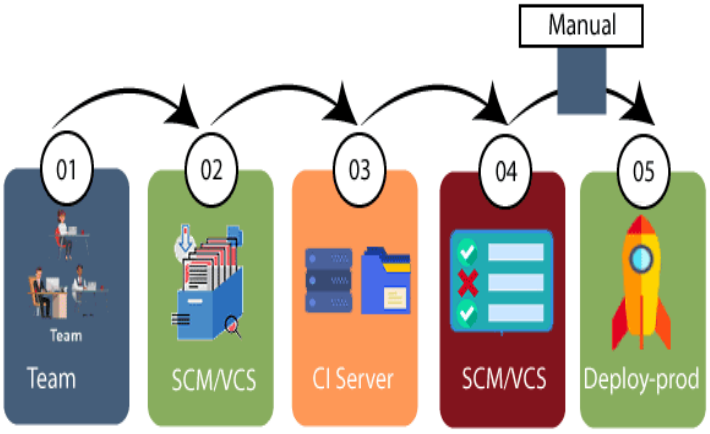


Figure 2.5: **Deployment Phase[9]**

2.3.4 Continuous Monitoring

Monitoring has become one of the phases that must be integrated in every stage of an application release and we will be referring to this phase in our project context chapter. Here most of companies seek to know how their application are performing during execution by seeing logs or exposed metrics, why is that ? well for the simple reason is automation

because we can use every metric for that specific application and try to auto-scale it by applying some rules with open-source tools like **prometheus** and wrappers.

2.3.5 Continuous Testing

As mentioned in the devops concept, the whole pipeline includes various testing phases wherein the automated tests are run along with the automated quality gates in between. We all know the importance of testing, testing every line of code at different stages and it is almost impossible to run all of the designed tests each time manually when a line of code is updated into version control.That's where the continuous testing comes into the picture.
So as defined earlier continuous testing execute various types of tests, continuously on the code base and on different environments as shown in figure 2.6.



Figure 2.6: **Testing Phase[10]**

2.4 Key elements of DevOps

Vorwaerts company has adopted several keys to ensure the success of the company in DevOps field. Such keys will decide the future of the company in the next years, In fact by integrating such company and understanding its foundations, many problems like poor communication between teams and cooperation between software development and operations reviled to be a huge challenge and issue at the same time[9] [10].

So to solve those issues leaders must follow up certain keys in order to decompose problems in different items and we can list them as shown in this table:2.1.

Element	Description
Automated Delivery	Defining the whole process that must me executed from the first moment when it will be published to source control till the availability of the code to customers. This concept relies on pipeline method in which every success build will trigger the new job.
Configuration Management	It is a way to ensure that the application only talks to its specific resources in the environment that is running on.
Monitoring And Health Check	This method allow developers or DevOps engineer to periodically and automatically check the health of the application before making its release to the customer and if so monitoring application will make it easier to see and report problems.
Sharing Culture	This approach is based on collaboration and sharing information among team,shifting responsibilities and creating a sense of shared duties between development and operations.

Table 2.1: Key Elements (characteristics) of DevOps

Conclusion

DevOps culture seems to be widely spread among companies and its adoption is quit challenging for most of developer in order to be acquainted with new technologies, methods and tools, parallel to performing their daily tasks of developing new software features.

Chapter 3

State Of Art

Introduction

This chapter will cover the most important things about the company like architecture adopted, tools used for pipelines and we will criticize the existing in order to introduce the solution.

3.1 Cloud Design

One of the benefits of cloud computing is the opportunity to swap upfront capital infrastructure cost with low variable costs that scale with growth. Each company will spend a huge time and hire experts so they can build their own infrastructure and solutions.
To understand better Vorwaerts infrastructure, we will talk first about the existing cloud solutions then we will move to describe ours.

3.1.1 Study Of The Existing

3.1.1.1 Amazon Web Service

AWS provides scalable and highly reliable infrastructure for deploying web/platform based solution of many business companies with minimal administration support.

AWS has far more services and features within these services than any other cloud provider, from infrastructure technologies such as computing, storage and databases, to emerging technologies, such as machine learning and artificial intelligence, data lakes and analytics and the Internet of Things. Thus, transferring your existing applications to the cloud and creating almost anything you can imagine is faster, simpler and more cost-effective[11].
This table 3.1 illustrate the different features and limits in AWS.

Features	Limits
• Friendly UI Design. • Consistent Documentation. • No Capacity Limits.	• Expensive Technical Support.

Table 3.1: Features And Limits of AWS

3.1.1.2 Microsoft Azure

Microsoft provides support for public, private, and hybrid clouds, it is a simple windows sever-based platform that allow developer to use any language, infrastructure tool to manage their applications[12].
This table 3.2 illustrate the different features and limits in Azure.

Features	Limits
• Good Scalability Options. • Strong Security Profile.	• Complicated UI Design. • Must Have Platform Expertise.

Table 3.2: Features And Limits of Azure

3.1.1.3 Heroku

Heroku is a PaaS platform supporting several languages for managing container systems to ensure deploying and managing modern applications. It offers for developer its own runtime system, expending application with add-ons while troubleshooting and remediation of common issues and customizing their ops experience[13].
This table 3.3 illustrate the different features and limits of Heroku.

Features	Limits
• An app-centric approach for software delivery. • Free Plan.	• larger applications tend to be very slow.

Table 3.3: Features And Limits of Heroku

3.1.1.4 IBM Cloud

IBM is one of the cloud providers that includes platform as a service and infrastructure as a service enabling organizations and large enterprise to Create and manage efficiently your applications.
The following table 3.4 will describe the different features and limits of IBM [14].

Features	Limits
• Organizations can determine their level of control. • Provides the most innovative technology.	• Lack of security.

Table 3.4: Features And Limits of IBM Cloud

3.1.1.5 Google Cloud Platform

Google Cloud Platform is google solution for cloud management, it is widely used and offers several services that each company is looking for. Those services like computing, data storage, data analysis and even ready machine learning algorithms allows users to have better understanding of their own applications architecture.
GCP offers PaaS , IaaS and serverless computing environments.
The following table 3.5 will describe the different features and limits of GCP [15].

Features	Limits
• Best Documentation. • Many Region To Store Data.	• Expensive fees. • Complicated UI Design.

Table 3.5: Features And Limits of GCP

3.1.2 Critics

As discussed above, Those are some cloud solution that offers services to many customer in the cloud but their features have limitations. For example our company used Heroku to deploy their own applications but this provider doesn't offer freedom when we talk about application management and has services limitation other than application deployment.
For that reason Vorwaerts has developed its own architecture to make application more reliable and orchestrated using microservices and devops tools to provide necessary resources also when it comes to monitoring it is more reasonable to use open source project and customize them as needed so it is better to the company to develop its own infrastructure.

3.1.3 Proposed Solution

To adapt more quickly to the changing market conditions, the solution is to use our private cloud instead of Heroku to host SaaS application mentioned in the previous chapter (

VAPA,VARAO,CDP) and use DevOps culture to enable better communication between teams.
As we all know that microservices applications are the future to ensure innovation, since developers can experiment on applications without fear of causing problems elsewhere and it is a necessity to better orchestrate application when it comes to IaaS.
One of the main goals that Vorwaerts focused on, using kubernetes to build flexible and reliable SaaS application and enabling open source project to interfere in the application architecture.

3.2 Cloud Architecture

Cloud architecture refers to the various component put together for converting a project's technical requirements into the architecture and design that will guide the final product. Often, cloud architects are also responsible for bridging the gap between complex business problems and cloud solutions. Other members of a technology team, including engineers and DevOps developers, work with the cloud architect to ensure that the right technology is being developed.

3.2.1 Hardware Component

The private cloud of Vorwaerts consists of ten servers provided by OVH [16] which is a French company who offers professionals infrastructures and software dedicated to data management, security and evolution.
OVH group claim to offer private and public cloud as well as shared hosting, DNS registrations, access via ADSL,VDSL,SDSL, fiber lines and Voip.
This provider has its own infrastructure and software strategy to satisfy their customers, in fact OVH cloud serve more than one million customer worldwide relying on a network of twenty datacenters across Europe, North America and Asia-Pacific.

3.2.2 Storage Component

For storage persistence our company chose to use Amazon S3 dedicated servers.Amazon Simple Storage Service (Amazon S3) is an object storage service that delivers scalability, data availability, security and state-of-the-art performance. It stores and protect any amount of data for a wide range of use cases such as websites, mobile applications, backup and restore, archiving, enterprise applications, IoT devices and Big Data analytic[17].
So our company uses some Amazon S3 benefits to back up data and the reason is that Amazon S3 service provide the following :

- **Low Cost**: Only pay for the amount of storage you use.
- **Security**: Amazon S3 offers flexible security features to block access to your data from unauthorized users.Amazon S3 supports both server-side encryption (with three key management options) and client-side encryption for data uploading.
- **Availability**: amazon S3 is designed to provide 99.99% data durability as this service automatically creates and stores copies of all S3 objects on multiple systems.
- **Simple Data Transfer**: Amazon S3 make data transfer effective and simple through a web service interface.

3.3 Network And Virtualisation Architecture

The main purpose for the company is to maintain its own architecture, manage networking and virtualization. This allows developers and engineers to test software that is under development or in the simulation stage.
In order to make this happen and ensure that all microservices applications are scalable and reliable three fundamental technologies are used : **Docker**, **Kubernetes** and our friendly UI manager **Rancher**.
We will proceed by talking about docker and what are the benefits to use containers in large companies then we will present the most powerful orchestration tool, kubernetes, and finally how to simplify the management of our application using rancher.

3.3.1 Docker

Before getting into why using docker for large companies and especially for ours, we must understand the reason behind it and clarify some terms, for that we will talk about the basic things that made docker so popular and study its architecture.

3.3.1.1 Basic Concept

Docker is the most largest software container platform used ever in software, cloud and development companies and according to ZDnet platform, over 4 millions applications have been placed in containers using Docker technology and over 37 billion containerized applications have been downloaded[18].
To make docker definition simple, we can say that docker is a technology that allows developer to package applications and all its dependencies together in the form of containers. The reason behind it is to make sure that the application works seamlessly in any environment which can be development or test or production.

3.3.1.2 Docker Fundamentals

As we mentioned before docker uses container technology that exists even when linux was created and those containers can be build while creating images pulled from private or public registries, so we will explain each term.

1. **Container**: Containerization is a virtualization concept or method that creates multiple units names as containers that shares the same host kernel in isolated manner through private namespaces and resource control mechanisms.
   Some advantages of containerisation are the followings :
   - Isolate applications from each others.
   - Run virtually anywhere with an easy development and deployment.
   - Easily switched from infrastructure to another.
2. **Images**: Any container created or updates is based on an image. An image is composed of multiple layers based on a Dockerfile that package up all application dependencies uch as code, config files, environment variables, libraries and run time.
3. **Registry**: Registries are like a storage concept for docker images that allows comapnies to host their own pre-build applications layers.

3.3.1.3 Docker Architecture

Docker is using a client-server architecture with three major components as follow:

- **Daemon**: Which is a server with persistent process that manages containers.
- **REST API**: Specifies interfaces that programs can use to talk to the daemon.
- **Docker Client**: Which is a CLI tool in order to interact against REST API to make requests.

The figure 4.3 describes docker architecture.
As we understand from docker architecture and how it works we can say that we can throw requests to a remote Docker daemon, but in large companies, this will cause problems especially for administration system team and developers.
From that idea came orchestration methods and tools like Kubernetes, docker swarm and many others but we will be interested just in kubernetes because it's the tool used in our company.

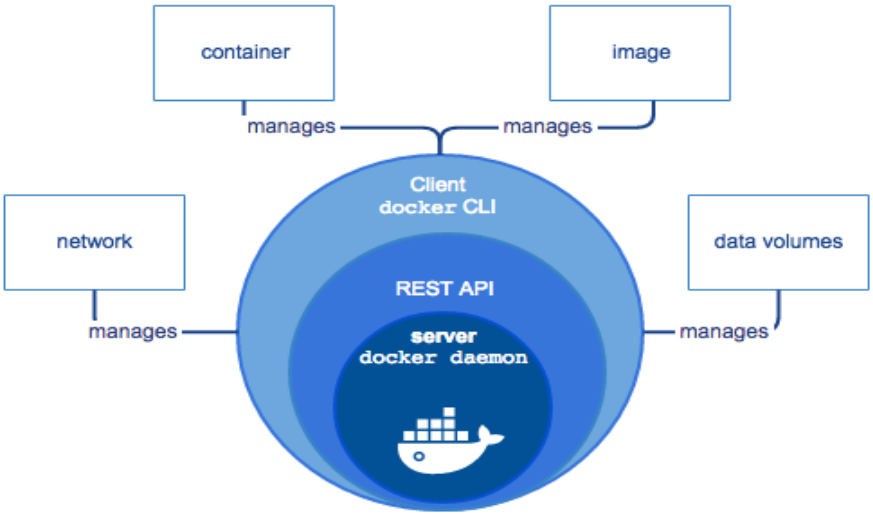


Figure 3.1: **Docker Architecture[11]**

3.3.2 Kubernetes

In order to make containers scalable and automated by a running engine, our company chose to use its own kubernetes cluster configuration in order to manage their tools over OVH cloud.
We will walk through an overview about kubernetes and what is made for, then we will refer to some kubernetes components to see how the architecture is build.

3.3.2.1 Overview

Kubernetes is an open source project developed at first by Google then was donated to CNCF. Till now it is the most efficient container-orchestrating system for managing containerized workloads across a variety of distributed clusters [19].
Because containers are low level entities, they require a framework to abstract those management configuration and automation scripts and provide all the additional features we expect from services deployed to clusters.
The reason of building such a tool is so simple from a containers perspective. As we mentioned before that containers need to have that on top framework in order to provide production features, fault tolerance is not provided by those running containers.

It means that if we want to operate containers at scale, be fault tolerant and self-healing, we need more than a scheduler like kubernetes and to answer the question why kubernetes, the benefits are the following[20]:

1. **Service discovery and load balancing**: Kubenrnetes can expose containers by using DNS controller to facilitate incoming and out-coming traffic routing across all containers with load balancers.

2. **Self-healing**: Kubernetes is able to restart damaged containers or those who do not respond to user-defined health check, and doesn't advertise them to clients until they are ready to serve.

3. **Storage orchestration**: Kubenretes allows users to automatically mount storage using Storage class yaml files.

4. **Automatic bin packing**: Kubernetes controls CPU and Memory of each nodes, containers and manage how containers can fits those requirements.

5. **Automated rollouts and rollbacks**: Kubenretes allow you to describe your own desired states of deployments and adapt all resources to the new controlled ones by using old or newest version of deployed containers.

6. **Secret and configuration management** : Kubernetes has the capability to store sensitive information and secret keys in configurations file and update them without rebuilding the whole architecture of containers.

3.3.2.2 Kubernetes Components

The term "cluster" refers to a functional deployment of Kubernetes. A Kubernetes cluster consists of two main components: the control plane and the computing machines or nodes. Each node is its own Linux environment. It can be a physical or virtual machine. Each node runs pods, consisting of containers. The following figure 4.5 resumes the interaction between control plane and working node which we are going to talk about [21].

By default in kubernetes cluster we can find a master node and one worker node but for high availability we can use a multi cluster with one or more worker nodes.
The master node components are the following[21][22]:

1. **ETCD**: Etcd is a distributed , reliable and consistent key-value storage mechanism used for service discovery and configuration management. It stores kubernetes cluster states and data such us number of pods, namespaces and API object details.

2. **Kube-Apiserver**: kube-apiserver exposes kubernetes API to processes REST operations, ensure their validation, and updates objects in etcd. It can be called as the front end for the Kubernetes control plane.
   This figure 3.2 resumes kube-apiserver concept when deploying new pod.

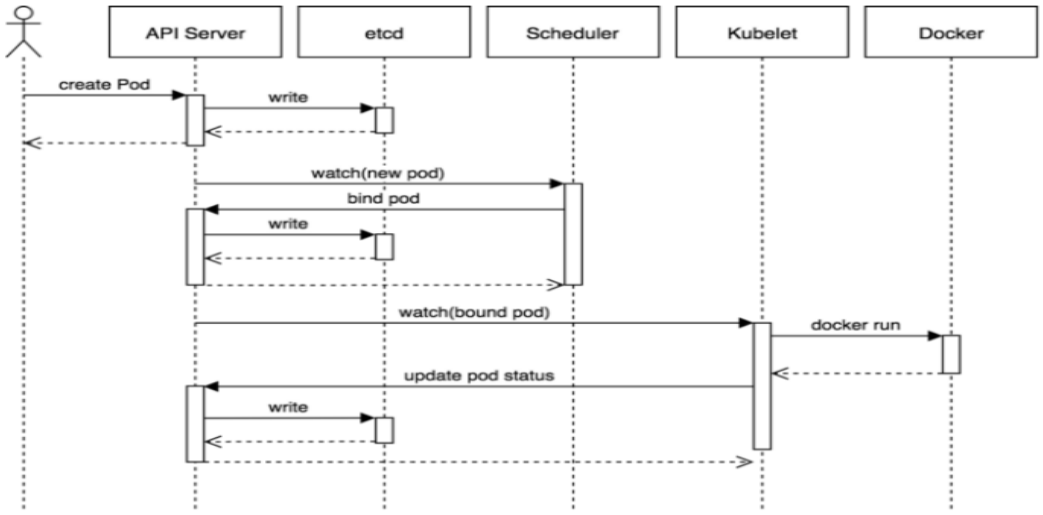


Figure 3.2: **Pod Creation and API concept[12]**

3. **Kube-Scheduler**: The scheduler watches over the newly created or unassigned pods and tries to bind them to nodes depending on their availability requested resources or / and quality of service requirements.

4. **Kube-Controller-Manage**: The controller is responsible for notifying user if a node goes down, ensure the number of desired pods and replications and creating the necessary features for newly namespaces created. This is done by referring to the ectd and look for new object stored taht are visible by the API-Server, finally parses the object and adds to the workqueue for it to be created.

For a simple worker node, we can find the following components:

1. **Kubelet**: It is an agent that runs on each node and make sure that the containers are healthy and running on each pod.

2. **Kube-Proxy**: It is a network proxy running on each worker node and make sure network communication to Pods from network sessions inside or outside of your cluster.

3.3.2.3 Kubernetes Deployment Kinds

When having a running kubernetes cluster, we can deploy our containerized application and to do so we have to instruct kubernetes with some abstraction concepts that shown below.

1. **Namespaces**: Namespaces Are like a virtual wall between multiple cluster because it specifies unique cluster resources per each name without having a collision when we have multiple access from different teams in projects located in those namespaces.

2. **Pods**: A pod is the smallest unit in kubernetes perspective. It encapsulates a number of containers that shares the same storage resources, provides a unique network IP and describes how containers must act in the cluster.

3. **Deployment**: Deployments are the upgraded version of ReplicaSet which provides us a perfect manner to deal with our running pods by describing how replicas we want and how those pods must run. They are used to ensure the availability of running pods, perform application updates without downtime and also the capability of rolling back to the previous containers version.

4. **Service**: Services represents an abstract way to expose application running in a set of pod as a network service allowing pods to communicate between each others without a need to keep track of which physical pod or which container make up the service.

   In kubernetes we have different type on how we want to describe the service like ClusterIP,Node Port and LoadBalancer.

5. **Volume**: Volumes solves the need to preserve running container states and files in case of crush. From kubernetes perspective there are move than 25 type of volumes and that is a reason why it is so popular, in fact those volumes are references directories made accessible to containers that form a Pod.

We have a lot of different concepts in kubernetes that we cannot list them all but as a brief view we can find Ingress, Secret, Config Map ,StorageClass and many others.

3.3.3 Rancher

As we mentioned before, Rancher is our tool used for managing our kubernetes cluster. As matter of fact, Rancher, as an open source software, enables companies to manage a multi-cluster production servers that relies on kubernetes and docker as a running engine. Rancher provides a significant values on top of kubernetes by provision compute nodes, centralizing authentication and role-based access control (RBAC) for all users to allow faster administration, enables cluster monitoring based on collecting metrics from servers and uses grafana for visualisation. Also it allows easier management of the cluster because it includes a pipeline engine to help automatically deploy and upgrade workloads [23].
To summer up , Rancher provides a friendly interface to help DevOps engineers to manage their applications workload without having an in-depth understanding of kubernetes architecture. Below are some features offered by Rancher:

1. **Infrastructure Management**: Rancher simplifies the creation of a kubernetes cluster through a friendly UI and its management by role-based access control for the hosted cluster.

2. **Monitoring**: Monitoring, health check and logging are the most important feature of Rancher because it keeps track of each metric exposed by the application.

3. **Users And Credentials**: Rancher is based on RBAC that manages permissions for any user who want to interact with cloud services.

4. **App Catalog**: Rancher offers the ability to deploy applications from its app catalog based on helm charts in order to make it easier for DevOps engineers to maintain applications.

The figures 3.3 presents an overview of rancher UI and how it integrates grafana for visualising cluster resources.

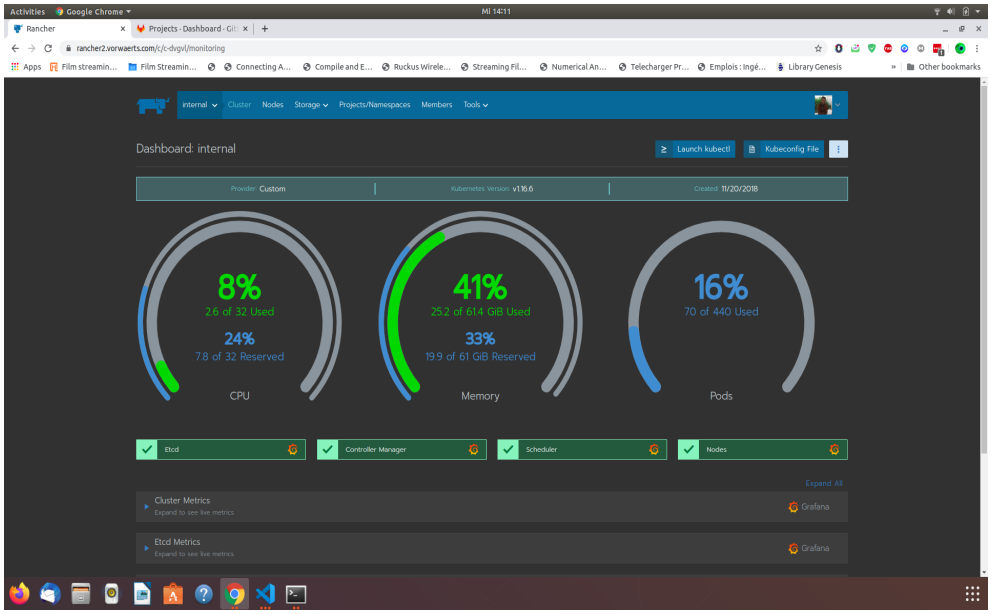


Figure 3.3: **Rancher UI Overview**[13]

Conclusion

In this state of arts chapter, we have talked about cloud design of the company and the stacks that we are using by mentioning different benefits of the type of cloud used also how infrastructure is based on. Till now we prepared the environment for integrating automation concepts in applications life-cycle that will be presented in next chapter.

Chapter 4

CI/CD pipeline Tools

Introduction

In this chapter we will devote the presentation to requirements analysis notion because it is an essential step of any It project. We will start by specifying technical goals that we must achieve, then adapting DevOps tools to our needs for this project and which tool is more useful in our state and finally talk about how to monitor applications with open source projects in a CI/CD pipeline.

4.1 Pipeline Architecture

4.1.1 Goals

Through our project we set up some goals that are defined below :

4.1.1.1 Technical Goals

- Technical documentation on Gitlab CI/CD.
- Technical documentation on OpenEBS.
- Technical documentation on Prometheus.
- Technical documentation on several projects like Bull exporter, IP failover, Nodejs.

4.1.1.2 Functional Goals

- Project configuration in Gitlab.
- Prepare deployments and pre-requisites for the platform.
- Pipeline implementation.

4.1.1.3 Product Backlog

The SCRUM approach helps teams to enumerate customer requirements in order to produce the product backlog as ”user stories” list.
The list below in table 4.1 describes everything the product requires designed for the project with the order that we set by product owner.

Number	User Story	Sprint
1	Host source code on Gitlab and define branches.	1
2	Prepare Deployments and configuration files.	2
3	Implementing Pipeline.	1
4	Monitor application and integrating Prometheus/EFK stack.	2

Table 4.1: Product Backlog

4.1.2 DevOps Tools

In this section we will be detailing some of the tools we used during our project by detailing their benefits and reasons behind choosing such softwares.

4.1.2.1 GitLab

4.1.2.1.1 Overview

Gitlab is an open source platform to manage DevOps lifecycle delivered as a software and change the way development, security and Ops teams collaborate and build software.
In other perspectives Gitlab helps team to reduce production cycle time, process and costs while increasing developer productivity.
So for many reasons, we can see that gitlab is widely used now in companies because it brings so many features that any team needs to add more reliability to their codes and among the features we can list :

- Ensure visibility and insight into how application is performing.
- Planning tools to keep teams synchronized.
- Create, view, and manage code through powerful branching tools.
- Keep quality standards when developing applications with automatic testing and reporting.
- Create a consistent software with built-in package management.
- Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST).
- GitLab integrated zero-touch code shipment.
- Automatically monitor metrics to be warned of any change in code impacts your production environment.

The figure 4.1 below summarize why gitlab must be the CI/CD tool for every company and how it acts in many different status for code production.

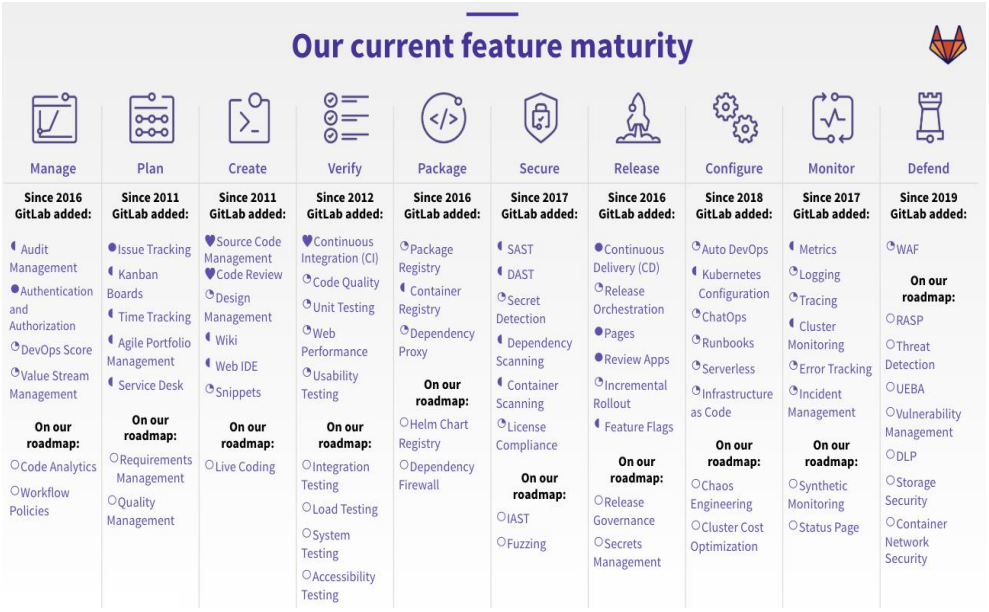


Figure 4.1: **Gitlab Features**[14]

4.1.2.1.2 CI/CD

Gitlab CI is the first part of code pipeline for each project, it builds and tests the application whenever testers or developer team pushes code. The second part is Gitlab CD, it is a kind of software service that integrates the changes of every code in the production state. Those two part are made through a script specified in a file called .gitlab-ci.yml, located in the root path of a project repository.

The most important thing is that Gitlab CI/CD tracks each step of the entire integration and release process without the need of any external other pipeline tools as shown in this simple example figure 4.2.

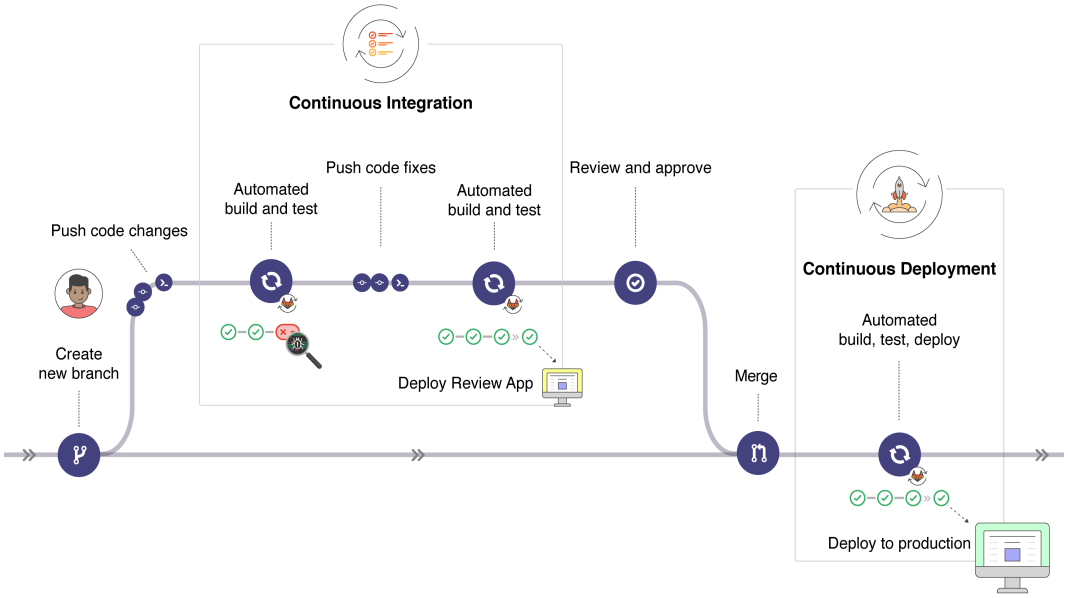


Figure 4.2: **Simple Gitlab CI/CD[15]**

4.1.2.2 Kaniko

We mentioned before that our environment depends on docker to build containers and images for our private applications, also we specified that while building a docker image through Dockerfile, the request is forwarded to docker daemon who will perform all the actions but this step always requires some root privileges that must be defined and configured.

The solution that is provided is using the Google open source project Kaniko. Kaniko works similarly to docker build but the main difference is that does not rely on docker daemon, so it make it easier for DevOps team to automate, resolve and build for each Dockerfile layer which allows pods to simply pull, build and push without those privilege permessions [24].

4.1.2.3 Helm

Before talking about Helm, its benefits and why using it will be more beneficial for kubernetes management, we must understand the main problem that comes after the development of companies architectures and the complexity of their applications [25].

Since the release of kubernetes, developing a single small application can result in a bunch

of detailed yaml file and configurations, so it will consume a lot of time and can affect production through multiple bugs and errors in the kubernetes cluster.
When it comes to solving such a problem we can talk about Helm, Helm is a package manager for Kubernetes allowing Dev and Ops team to easily package, configure, and deploy their own charts onto Kubernetes clusters. Helm is made of two component:

- **Tiller Server**: Held inside the kubernetes cluster and interacts with helm client. It is responsible for installing, upgrading, deleting and translating charts to and from kubernetes API.
- **Helm Client**: Helps developers to create and manage their chart repositories.

The figure 4.3 below describes helm components interaction in its architecture.

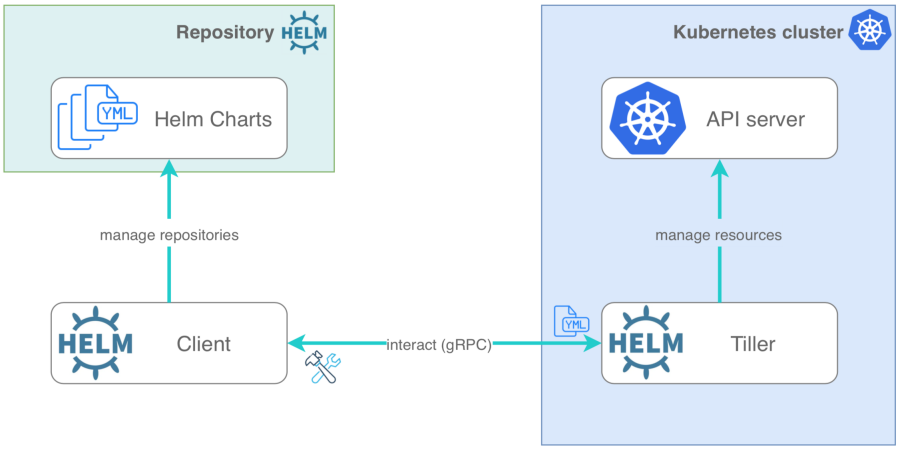


Figure 4.3: **Helm Architecture[16]**

4.1.2.4 Cert-Manager

As we now that security matters for all companies that hosts different platform or applications through certificates and key encryption, so from kubernetes layer we used Cert-Manager for this issue.
Cert-manager is a kubernetes certification management controller to help issuing certificates from different sources such as Venafi, HashiCorp Vault and Let's Encrypt.

Cert-manager make sure that certification are updated and valid for each period before they expire.

4.1.2.5 OpenEBS

4.1.2.5.1 Overview

OpenEBS [26] is an open source project used widely now for kubernetes storage related to container-native and container-attached storage (CAS) type. It implements policies and isolation to optimize storage for each specific workload regarding users perspective. What CAS provides for user is the facility of using storage entities because it is divided into two separated components, data plane (collection of Pods) and control plane based on CRD to manipulate storage entities.
Choosing OpenEBS as a provisioning concept for implementing storage into kubernetes resources makes life easier for DevOps team to manage their storage workloads and dynamically provision volumes without rules or permission as different way from other storage solution.
The answer of the why question are defined here :

- OpenEBS has a microservice architecture and manages its components (pod, services, deployments, configs) on kubernetes cluster.
- OpenEBS has a wide range of storage engines that can help users to choose the better one for their workloads.
- OpenEBS provides a good manageability, snapshots, availability, and clones.
- OpenEBS has many add-ons that integrates with kubernetes restore and backup mechanism.
- OpenEBS offers high availability because it follows the CAS architecture, so it makes it easier for kubernetes scheduler to reschedule its componenets while the data is protected by the use of replicas.

When it comes to persistent volumes that must be attached to each container, OpenEBS has its own mechanism on how doing it.

We chose to use Local Pv provisioning so the figure 4.4 below will describes the workflow of dynamically provisioning a volume using OpenEBS project.

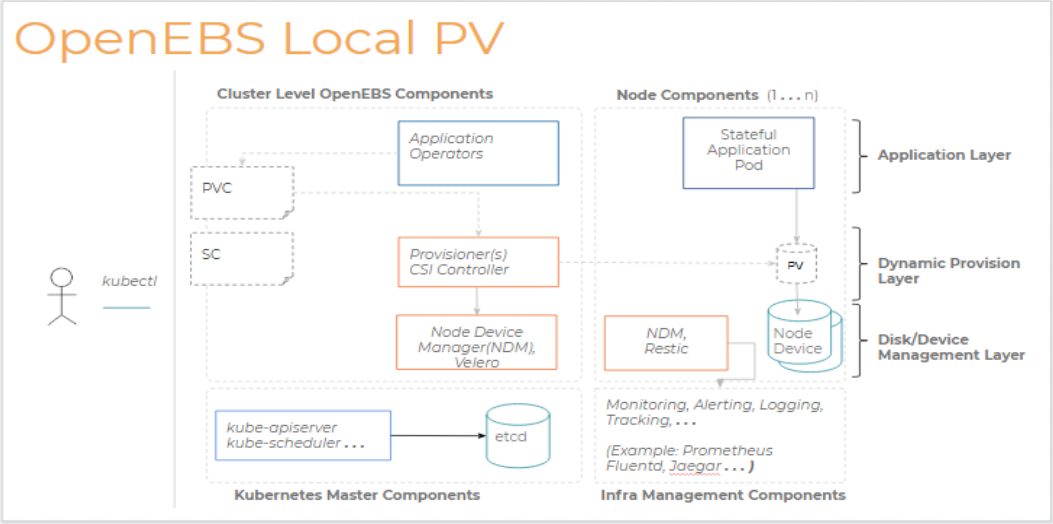


Figure 4.4: **OpenEBS Local PV Provisioning[17]**

4.1.2.5.2 Release

We will proceed the installation of this project referring to official documentation through those steps:

1. **Set cluster-admin user context** : kubectl config set-context $(kubectl config current-context) –namespace=ebs .
2. **Installation through helm3**:
   - helm repo add openebs https://openebs.github.io/charts
   - helm repo update
   - helm install openebs –namespace ebs openebs/openebs
3. **Verifying OpenEBS**:

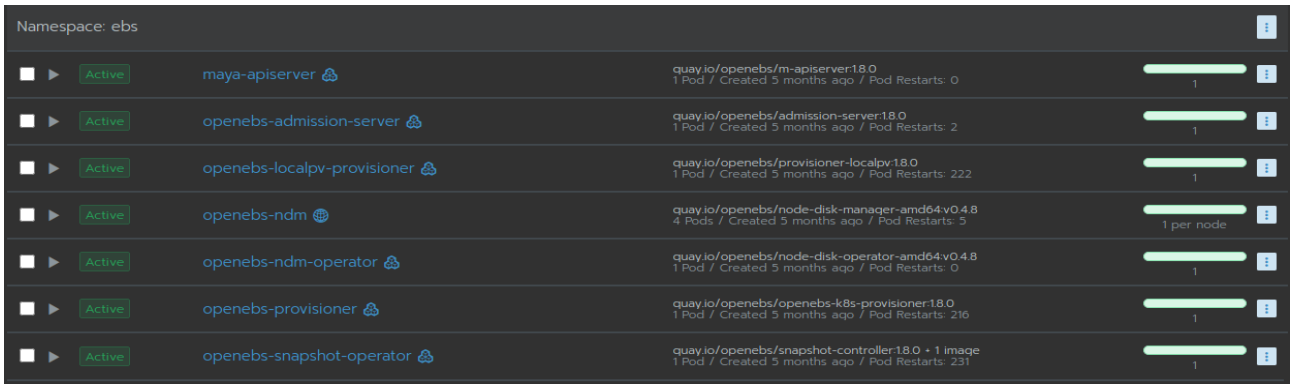


Figure 4.5: **OpenEBS Installation[18]**

4.2 Monitoring Concept

Monitoring and logging are the crucial steps that any developer must understand to anticipate problems and discover bottlenecks in a production environment. With the growth of microservices applications, we saw that logging and monitoring is becoming a complex concept because of the communication between variety of distributed services written in several languages.

We can find many tools build for those purposes but we chose to use open source projects and integrate them with our private cloud, so we will explain in details each tool used in this section.

4.2.1 Open Source tools

4.2.1.1 Prometheus

Prometheus is an open source project based on monitoring and alerting through API. This project attracted so many developer and companies so it was adopted by CNCF to increase its community as an efficient open source project available for all monitoring level [27].

Choosing to use prometheus is the best solution for our company because many kubernetes objects ship Prometheus-format metrics by default, so it will be easy to explore and detect such metrics with prometheus and by deploying it into our cluster we can benefit from :

1. **Multi-dimensional data model**: Prometheus is based on key-value pairs and a time series storing concept similar on how kubernetes acts.

2. **Flexible Query Language**: Prometheus integrates PromQL to select, query metrics collected preparing them for subsequent analysis and visualization.

3. **Easy Metrics Collection**: Prometheus allow users to collect and detect metrics automatically from application by just exposing them on different ports by designing API endpoints.

4. **Service discovery**: Metrics are automatically pulled from application periodically by discovering each endpoints or new port added. It uses auto-discover method to scrape designed metrics.

Many releases are developed by prometheus community in order to adapt to many changes of how applications are performing, for that reason so many features and sub-projects were added and we are going to discuss them based on our needs.

4.2.1.1.1 Prometheus Operator

Prometheus operator is also a project developed under the prometheus community hosted by CoreOS, the main objective of this pattern is make it easier for operation team to manage prometheus instance in the kubernetes layer and external application services that our application exposes [28].
To see how prometheus operator works, we will describe how we have proceeded through those steps:

1. Installing and configuring the Kubernetes-Prometheus stack.

2. Auto-configuring VARAO endpoints with ”ServiceMonitor” entities.

3. Scale application services using CRD operator and ConfigMaps to make it fully portable and declarative.

We mentioned that prometheus operator relies on CRD, so we will define those custom resource definitions and explain the role of each one based on a simple architecture in the figure 4.6.

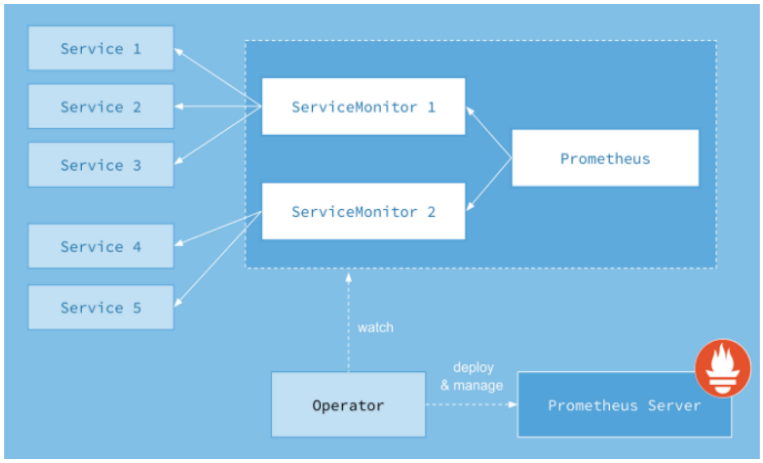


Figure 4.6: **Prometheus Operator Architecture[19]**

As we can see in this picture after installing the prometheus stack that we have many deployments like the Prometheus servers, kube-state-metrics and its operator.
Prometheus is always watching over kubernetes API, so when we will have a new service created or updated for an application, they will be detected automatically through a ServiceMonitor instance which are controlled by an operator to deploy new sets of configurations into the servers.
Just to define what is a ServiceMonitor, it is a group of instance which specifies how a set services should be monitored and for the Operator, it automatically generates Prometheus scrape configuration based on the definition.

4.2.1.1.2 Prometheus Adapter

One of the main idea in kubernetes deployments is knowing how to auto scale application replicas based on use-cases, so we decided to use prometheus operator for HPA (Horizontal Pod Autoscale) of custom metrics developed with JS by our Dev team.
Extending kubernetes functionalities through prometheus adapter is a suitable solution in our case because it helps us to leverage collected metrics and make scaling decisions.
Returning to our use case, VARAO application is developed to support customers needs as defined in the first chapter, in fact this platform has its own custom metrics published through endpoints but prometheus does not know those exposed metrics by kubernetes API, here were it comes prometheus adapter to not only provide prometheus with the ability to scrape custom metrics but also to make it easier to scale the application [29].

4.2.1.2 EFK Stack

For container logging and monitoring we chose to deploy EFK stack into our cluster in order to monitor and log each containers based in different nodes. In fact we may wonder why logging containers, indeed, for most system administrator it is easier to be up to date on each log of any container in order to see how it reacts within the cluster [30].
In this case why not using docker command "docker logs ..." ?, the main reason resides when you have multiple clusters running thousands of containers and communication with each other every seconds.
The solution was to host our private logging system based on Fluend. Fluentd is an open source project that unifies data collection and consumption by collecting, filtering, buffering, and outputting logs across multiple sources and destinations.
Based on the architecture of Fluentd in the figure 4.7 below, we can say that having a distributed components in the cluster to scraps logs from a given set of sources, processes them and then send those logs from docker containers in the "/var/lib/docker" directories is more efficient that logging to each one manually.

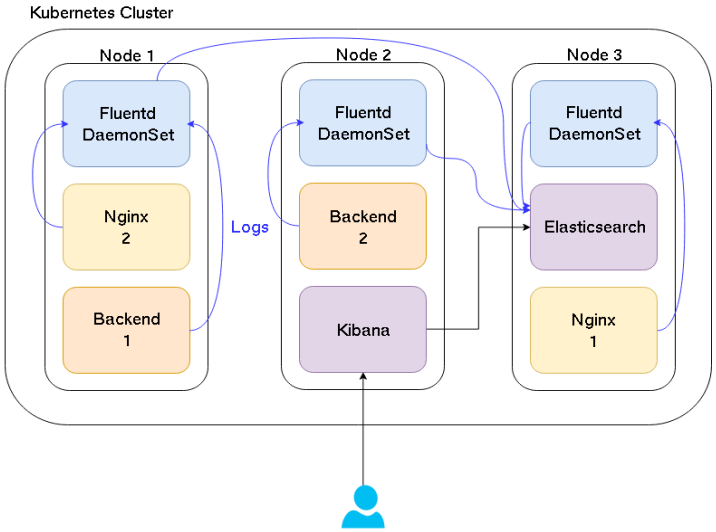


Figure 4.7: **Fluentd Mechanism [20]**

In the other hand, we have our elasticsearch which is a real-time, distributed, and scalable search engine to analyse all those logs transmitted by fluentd and store them in a common format and then we use our beautiful kibana dashboard to visualize our custom data as shown in figure 4.8.

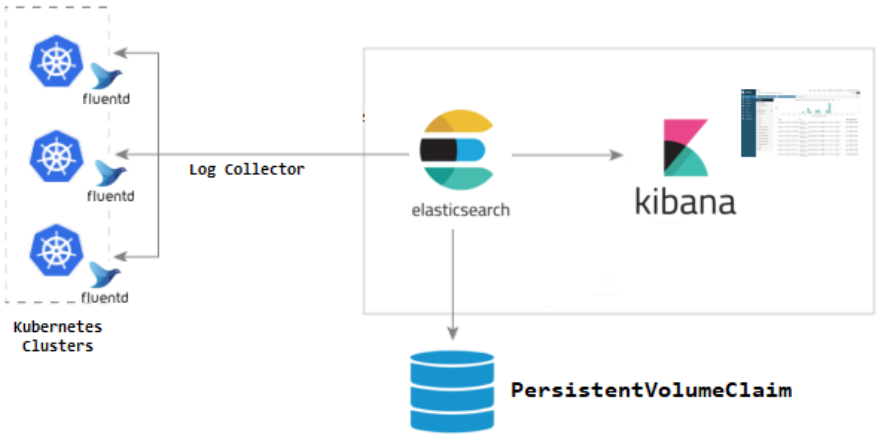


Figure 4.8: **EFK Architecture [21]**

4.2.1.3 Bull Exporter

Since our VARAO platform is based on redis which is an open source in memory data structure store used for caching and Mongodb for data storing, we may face a huge problem since we must have faster communication between microservices components [31].
Bull is an open source project designed for a fast and robust queue system based on redis. It is a nodejs library that takes care of the whole low-level details and enriches Redis basic functionality to robust communication channels between microservices based on API.

4.2.2 Horizontal Pod Autoscaling

4.2.2.1 Overview

Autoscaling is a trend concept in the life of any microservice application deployed on any orchestration solution to scale up or down workloads based on the resource usage. It was implemented as a loop to query metrics API periodically like CPU/memory and some pre-defined Custom Metrics API for application-specific metrics, in fact we can mention external/custom metrics that we explained before and offered through adapter by metrics solution vendors like prometheus [32].
Kubernetes includes several autoscaling features like :

- **Horizontal Pod Autoscaler**: Scales the number of pods in a deployment or replica set.
- **Vertical Pod Autoscaler**: Adjusts the resource requests and limits of the containers of an application.
- **Cluster Autoscaler**: Deals with node scaling operations.

We will be just focusing on HPA and as we said before, HPA continuously look over metrics app through the controller manager to adjust the number of replicas desired by the administrator. The figure 4.9 below shows how this control loop is functioning, first Query the scaling metrics then estimates the desired number of replicas based on system resources and finally scale the application by increasing or decreasing the number of pods.



Figure 4.9: **HPA Control Loop [22]**

The scaling concept is based on a simple yaml file that will be deployed all along with the application resources and that file includes the following parts:

- Deployment name of the application.
- Minimum and maximum number of replicas.
- The scaled metric that must be specified explicitly.
- Target value for the scaling metric.

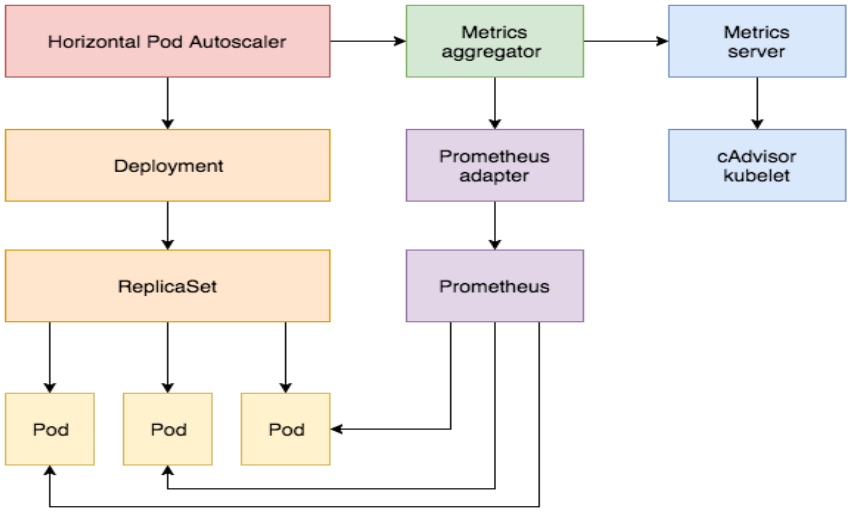


Figure 4.10: **HPA Architecture [23]**

4.2.2.2 HPA Architecture

The figure 5.13 show the whole architecture on how HPA is working with prometheus project, the details are below explained as steps format [33]:

1. The controller manager queries the resource metrics specified in each HPA yaml defenition. In this case we have resource metrics API which defines CPU and memory per each pod or custom metrics defined previously.

   - **Per-pod resource metrics**: Fetching metrics from each pod targeted by the HPA, then the controller calculates the resources used compared to node resources and estimates if it will face an increase or not. If so it scales the number of desired replicas.
   - **Per-pod custom metrics**: It works with the same manner of per-pod resource metrics except it works with raw values.

   If the application has its own endpoints the new scenario will be managed by prometheus.

2. The controller manager queries the custom/external metrics configured in the deployments by interacting with prometheus operator through REST API.

3. Prometheus collects metric data from the Pods through a ConfigMap yaml file where we can define the pod port or the explicit endpoints.

4. Prometheus Adapter obtains the collected metric data from Prometheus and serves it as a named metric through the Custom Metrics API.

5. The HPA queries the named metric from the Custom Metrics API and uses it to autoscale the application deployment.

Conclusion

All along in this chapter we prepared the ground for the realization part while defining the working environment, tools to be used and monitoring concepts.
The next chapter will be the final one where we present how we managed to combine all the explained methodology for better application production.

Chapter 5

Release

Introduction

In this chapter we will cover the release of our version ”2” of our VARAO application based on the architecture proposed in the previous chapter for each sprint.
The first step will be dedicated to the development of varao kubernetes charts using Helm, secondly we will configure gitlab and implement the CI,/CD pipeline to track each step of the deployment and make it easier for developer to make changes and finally we are going to configure monitoring concept not only for the application but also for the whole cluster.

5.1 Charts Preparation

5.1.1 Overview

The previous chapter covered the use of helm and cert-manager in our project. In fact those two tools will allow us to create our charts for the VARAO application in microservice architecture and add certification for security level.
After preparing the kubernetes resources for the whole application, we will use one of the helm specification, the ”values.yaml” file which is used to add values of the charts in a structured way so that we can make changes easier rather that updating all kubernetes resources deployed.

5.1.2 Charts Components

5.1.2.1 Containerisation

For any company that is investing in the IT market, in is essential to use containers to leverage the complexity of microservice component for better performance, for that reason we will create our own docker file for the application and host it in our private registry. After reviewing the application requirements we were able to create this simple docker file to package up all dependencies in the figure 5.1 below.

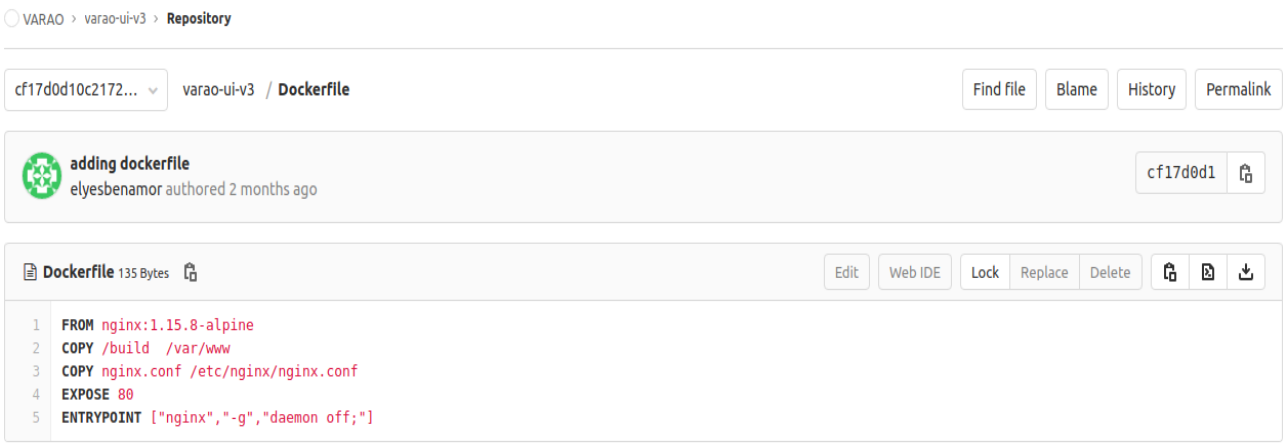


Figure 5.1: **VARAO Dockerfile[24]**

5.1.2.2 Kubernetes Deployments

5.1.2.2.1 Deployment

In this deployment chart, kubernetes will pull the image of VARAO application from the Vorwaerts private registry using ”imagePullSecrets” annotation, then will run multiple replicas as desired and automatically will replace any failed instance or become unresponsive.

The figure 5.2 below is the yaml file that will be consumed by kubernetes cluster.

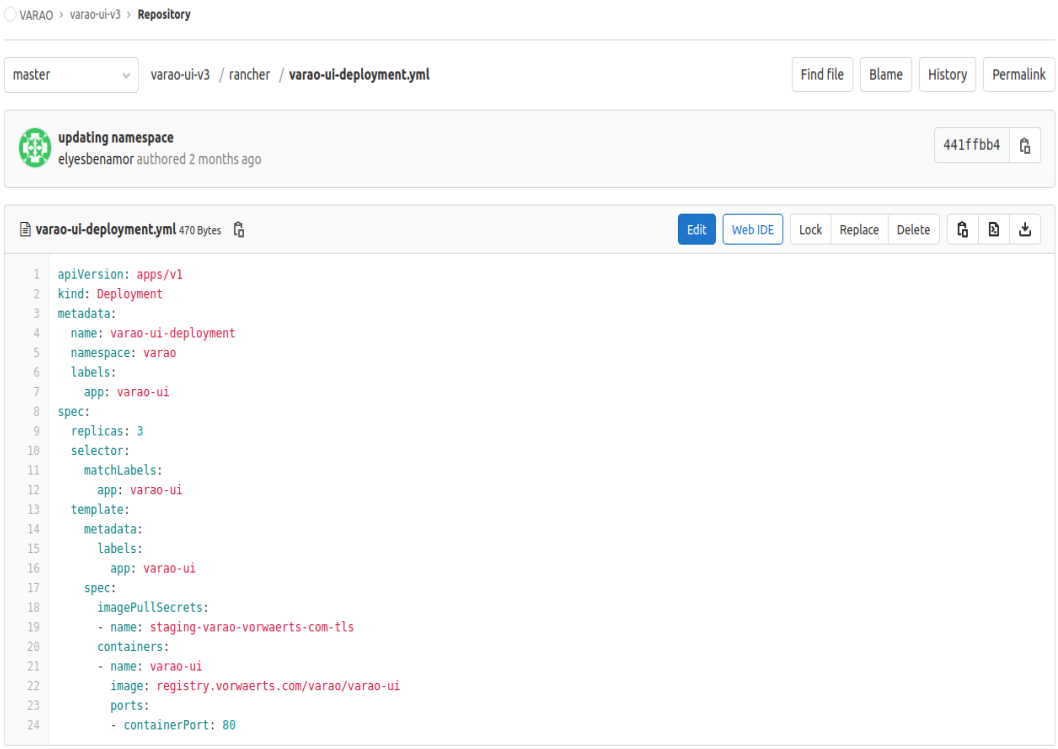


Figure 5.2: **VARAO Yaml File Deployment[25]**

5.1.2.2.2 Service

This specification creates a new Service object named ”varao-ui-svc”, which targets TCP port 80 on any pod with the label ”app = varao-ui”.

Kubernetes then will assigns this service an IP address (sometimes called the ”cluster IP”), which is used by Service proxies.

The service controller continuously searches for pods that match its selector, then POST all updates to an Endpoint object also called ”varao-ui-svc”.

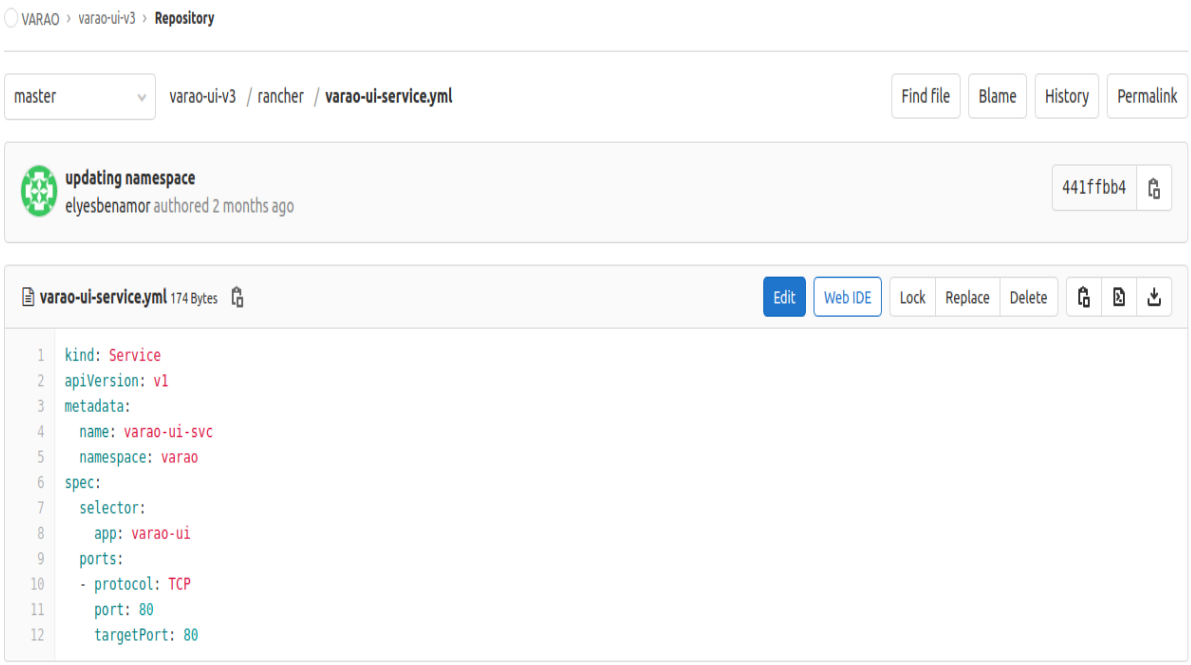


Figure 5.3: **VARAO Yaml File Service[26]**

5.1.2.2.3 Ingress

We will use ingress annotations to configure some options depending on the Ingress controller by applying a list of rules against all incoming requests. Ingress resource only supports rules for directing HTTP(S) traffic.

The configuration will be shown in the value file.

5.1.2.2.4 SSL Certificate

To make our application secure, we will add CA certificate bundle to the list of CA certificates that the TLS client or server trusts so that when request come from any user our controller will verify the certificate added by the help of the cert-manager.

As shown in the figure 5.4, we configured our certificate deployment with a secret name to assign automatically a certification the the host ”staging.varao.vorwaerts.com”.

VARAO › varao-ui-v3 › Repository

d4e837d571889... varao-ui-v3 / CI / sslcert.yaml Find file Blame History Permalink

adding ssl certeficate for varao ui
elyesbenamor authored 2 months ago d4e837d5

sslcert.yaml 1000 Bytes Edit Web IDE Lock Replace Delete

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: varao-ui

---

apiVersion: cert-manager.io/v1alpha2
kind: Issuer
metadata:
  name: selfsigned
  namespace: varao-ui
spec:
  selfSigned: {}
---
apiVersion: cert-manager.io/v1alpha2
kind: Certificate
metadata:
  name: selfsigned-cert
  namespace: varao-ui
spec:
  commonName: staging.varao.vorwaerts.com
  secretName: staging-varao-vorwaerts-com-tls
  issuerRef:
    name: selfsigned
```

Figure 5.4: **VARAO Yaml File SSL Certification[27]**

As we mentioned before using helm charts require the value.yaml file that has all the values for all dependencies.

In the figure 5.5 we will present a snippet of value.yaml code for our VARAO application.

values.yaml 569 Bytes

```yaml
image:
    repository: registry.vorwaerts.com/varao/varao-ui
    pullPolicy: Always

  revisionHistoryLimit: 2
  replicaCount: 3

  service:
    type: ClusterIP
    ports:
      - port: 80
        targetPort: 80
        protocol: TCP
        name: http

  ingress:
    enabled: true
    annotations:
      kubernetes.io/tls-acme: 'true'
    tls:
      - hosts:
          - staging.varao.vorwaerts.com
        secretName: staging-varao-vorwaerts-com-tls
    paths:
      - '/'
    hosts:
      - staging.varao.vorwaerts.com

  nameOverride: 'staging-varao-ui'
```

Figure 5.5: **VARAO Yaml File SSL Certification[28]**

5.2 Gitlab Pipeline

5.2.1 Design

As we explained before, Gitlab CI/CD is the most powerful pipeline tool e.g we can create or define as much stages as we want for our application to track and execute jobs in need. For example, as shown in the figure 5.7, we can see that Gitlab pipeline help you see in which stage you got failure and review jobs triggered for each stage.

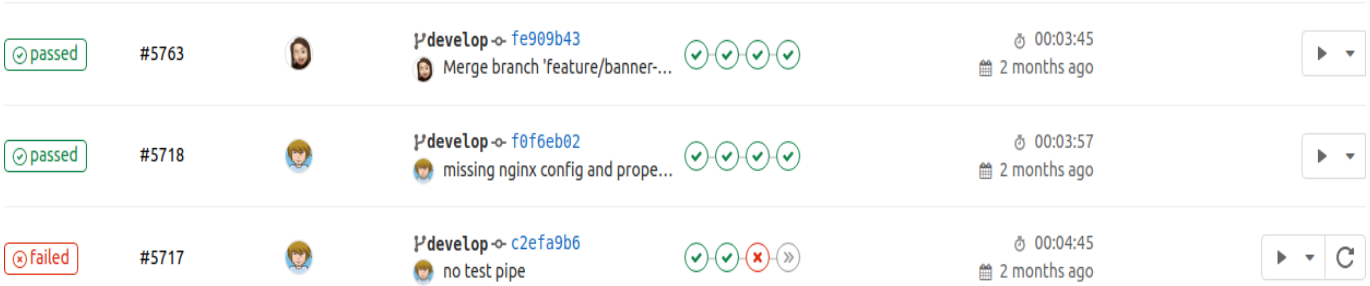


Figure 5.6: **Four Stage Pipeline[29]**

The pipeline we are going to implement has different stages that we are going to explain in order to see how is efficient to use such concept to deploy containerized application in production level.

- **Testing**: In this stage, a job will be triggered to verify if we have the necessary dependencies to deploy our application in the docker container image.
- **Fetching**: In this step, the pipeline will wait for any change that will occur when pushing new updates to the code, if so a job will take action to start retrieving code from directory.
- **Building**: This is the most important stage because we are going to byuild our docker image and push it to the private registry using Kaniko project.
- **Deploy**: Final step that helps the team to deploy application into the cluster for production.

5.2.2 Implementation

In this part we are going to detail the main steps for making our pipeline using ".gitlab-ci.yaml" file added to the root of the main directory, but first we have to configure Gitlab runner and link our project to the kubernetes cluster.

A Gitlab runner is a light worker machine that executes CI jobs for the project hosted in Gitlab. In order to execute jobs we will use as mentioned the .gitlab-ci.yml file where we will define our stages, variables and dependencies that will be used.

Vorwaerts private cloud has different environment like staging or testing cluster, internal cluster for testing our application before releasing them to the production cluster for our customers.

The figure 5.7 presents the UI interface for adding the project to the dedicated cluster.

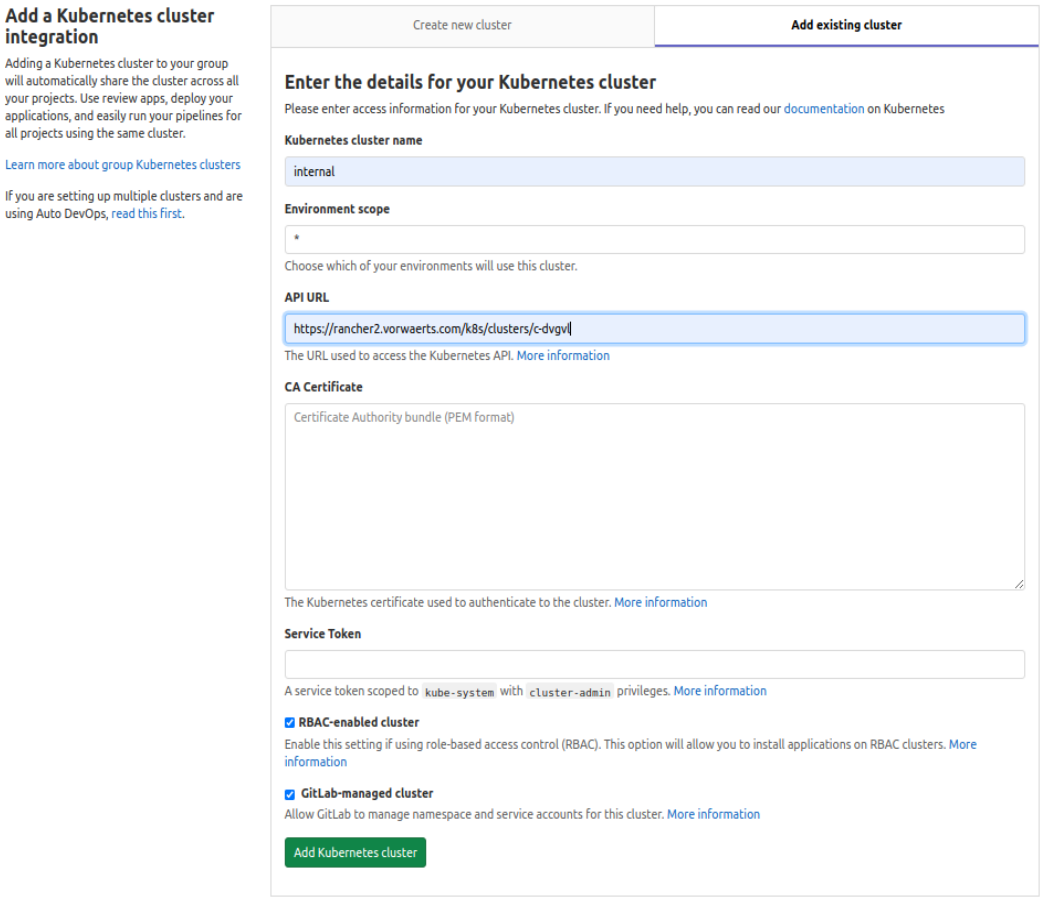


Figure 5.7: **Gitlab Environment Configuration[30]**

After configuring our environment and preparing the necessary deployments for our application it is time to create the pipeline file that will cover all the steps to ensure the

release of our application into the internal cluster.

In general Gitlab CI/CD runs on a Docker container, that is why in the beginning of the file we must specify which image to use to trigger jobs where they will be executed in a running containers.

When it comes to stages, we can see that we created multiple stages in order to make it simpler for DevOps team to track and debug any step without conflicts and as shown in the figure 5.8, we have a detailed description of the stages including how to define modules and scripts that we will use into different stages based on application need.

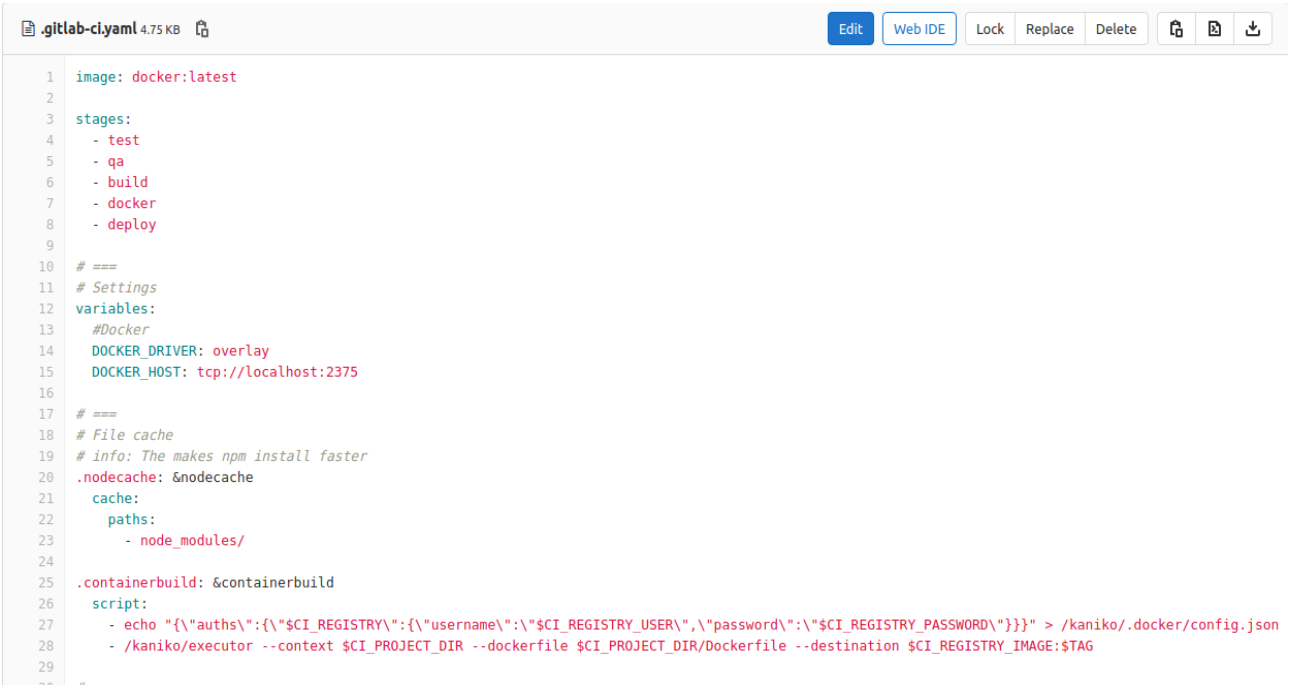


Figure 5.8: **Gitlab Stages And Modules[31]**

Now we will detail the stages mentioned previously so we can understand better how pipeline is running in Gitlab architecture.

5.2.2.1 Test

This step is introduced as testing because it first going to pull a node image from Vorwaerts registry and run the the necessary scripts to install packages for our application based on Nodejs as shown in figure 5.9.

```
31  # Testing
32
33  # Lint
34  tests::lint:
35    image: node
36    stage: test
37    <<: *nodecache
38    before_script:
39      - rm -rf package-lock.json
40      - npm install
41    script:
42      - npm run test:lint
43    allow_failure: true
44
```

Figure 5.9: **Gitlab Test Stage[32]**

5.2.2.2 Build

The second stage defined is building the image or it is also called Dockerisation. This is where we build our image using the Kaniko project mentioned before.
If we take a deeper look into the file for the build stage we can recognize that we have three different types of building, to define them in a simple way it is because we must differentiate each build for each git branch so we can avoid conflicts.
In other way, this will help us add a unique ID to each build container through those tags explained consecutively:

- Dev tag is used we we release new version to production cluster and it is created from the master branch carrying the last changes.
- Latest tag used for master branch where we have last software version.
- Commit tag generated with a random hash by gitlab for each new commit.

```
#Build
build:
  image: node:latest
  stage: build
  variables:
    STAGINGURL: https://staging.varao.gateway.vorwaerts.com
  before_script:
    - npm install
  script:
    - if [ "$CI_COMMIT_REF_NAME" = "upgradeToV2API" ]; then sed -i "s%^export const URL.*$%export const URL = '$STAGINGURL'%" src/constants/ApiConstan
    - npm run build
  artifacts:
    paths:
      - build
    expire_in: 5d

# ===
# Docker

# Build Docker with commit sha docker tag
build:container:sha:
  stage: docker
  image:
    name: gcr.io/kaniko-project/executor:debug
    entrypoint: [""]
  variables:
    TAG: "dev"
  <<: *containerbuild
  only:
    - upgradeToV2API

# Build Docker with commit latest tag
build:container:latest:
  stage: docker
  image:
    name: gcr.io/kaniko-project/executor:debug
    entrypoint: [""]
  variables:
    TAG: "latest"
  <<: *containerbuild
  only:
    - master

#build for tags
build:container:tag:
  stage: docker
  image:
    name: gcr.io/kaniko-project/executor:debug
    entrypoint: [""]
  variables:
    TAG: "${CI_COMMIT_TAG}"
  <<: *containerbuild
  only:
    - tags
```

Figure 5.10: **Gitlab Build Stage[33]**

The figure 5.10 presents the building part from the gitlab pipeline code. The main steps for building such an application are as follows:

1. Specifying the build tool Docker image which in our case is Kaniko.
2. Specifying tags for each branch.
3. Inject the script inside the container wihc is in our case ".containerbuild".

5.2.2.3 Deploy

Once the build is complete we will proceed to the next stage which is deployin our case. This stage where we deploy the containerized application into our cluster and as mentioned before we are going to do that with the help of helm to make the archive procedure.
In the deploy stage we also specify which container environment to use, in our case is a helm container that has all the helm packages, also in which namespace, environment and the scripts to use.
Referring to the build stage, if we build the code into the "upgradeToV2API" branch for staging then the deploy will also be the same because it has all the post-production dependencies.
The figure 5.11 presents the deployment part configuration while explaining each step:

1. Specifying the build tool Docker image which in our case is Helm.
2. Specifying the variable to use like namespace and where to deploy as configured when adding cluster.
3. Inject the script that will execute the deployment process.
4. Specifying the branches in which the process will be triggered.

```
137 # Deploy
138
139 #Staging
140 deploy::staging:
141   image: registry.vorwaerts.com/microservices/helm
142   stage: deploy
143   variables:
144     NAME: "$CI_ENVIRONMENT_SLUG-$CI_PROJECT_NAME"
145     PULLSECRET: $pullSecret
146     NAMESPACE: "varao"
147     #TAG: "dev"
148   script:
149     - export DEPLOYS=$(helm tiller run "$NAMESPACE" -- helm ls | grep "$NAME" | wc -l)
150     #- helm tiller run "$NAMESPACE" -- helm ls
151     #- helm tiller run "$NAMESPACE" -- helm ls | grep "$NAME"
152     #- helm tiller run "$NAMESPACE" -- helm ls | grep "$NAME" | wc -l
153     - if [ ${DEPLOYS}  -eq 0 ]; then helm tiller run "$NAMESPACE" -- helm install --name="$NAME" --namespace "$NAMESPACE" /charts/deployment -f CI/sta
154   environment:
155     name: staging
156   only:
157     - upgradeToV2API
158   when: manual
159
160 #TAGS
161 deploy::tags:
162   image: registry.vorwaerts.com/microservices/helm
163   stage: deploy
164   variables:
165     NAME: "$CI_ENVIRONMENT_SLUG-$CI_PROJECT_NAME"
166     PULLSECRET: $pullSecret
167     NAMESPACE: "varao"
168   script:
169     - export DEPLOYS=$(helm tiller run "$NAMESPACE" -- helm ls | grep "$NAME" | wc -l)
170     #- helm tiller run "$NAMESPACE" -- helm ls
171     #- helm tiller run "$NAMESPACE" -- helm ls | grep "$NAME"
172     #- helm tiller run "$NAMESPACE" -- helm ls | grep "$NAME" | wc -l
173     - if [ ${DEPLOYS}  -eq 0 ]; then helm tiller run "$NAMESPACE" -- helm install --name="$NAME" --namespace "$NAMESPACE" /charts/deployment -f CI/pro
174   environment:
175     name: production
176   only:
177     - tags
178   when: manual
```

Figure 5.11: **Gitlab Deploy Stage[34]**

After implementing the CI/CD pipeline for the whole deployment of the VARAO platform, it is time to add some efficiency to track and monitor the behaviour of the application. To insure that our system is handling properly all containerized application while having a continuously look for the hardware system, we have to implement a monitoring concept to ensure our costumers that our tools are secure and work as predicted.
The next section will be dedicated to the monitoring concept while specifying how to use prometheus and EFK stack in production environments.

5.3 Monitoring Configuration

5.3.1 Rancher Embedded Monitoring

Rancher is a powerful tool for tracking our cluster and easily deploy application through helm packages, but since the last upgrade we saw that it implements the prometheus project for keeping an eye over the kubernetes metrics and use those exposed ones to use them as data into Grafana dashboard.
Grafana is an open source project for data visualization, monitoring and analysis of CPU/Memory and network traffic.
The figures 5.12 and 5.13, below represents the embedded monitoring system in Rancher for our Nodes and VARAO application.

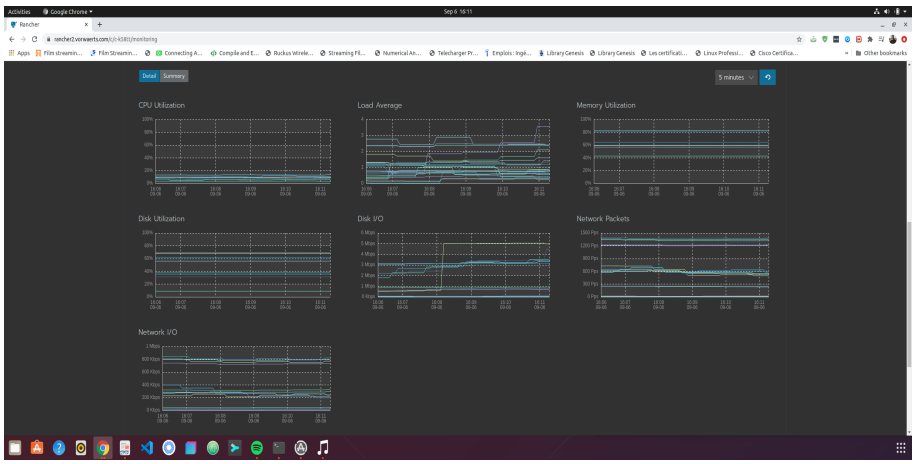


Figure 5.12: **Embedded Rancher Monitoring[35]**

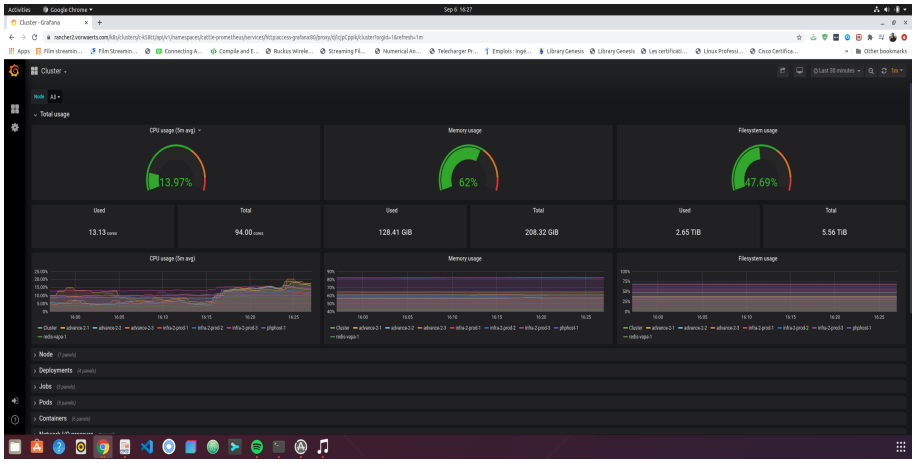


Figure 5.13: **VARAO Grafana Dashboard [36]**

5.3.2 EFK Container Logging

We have talked previously that we are going to integrate EFK stack for logging containers and tracking system failure.

The main solution was to use EFK charts and deploy it into our cluster using a persistent storage mechanism which is OpenEBS to dynamically allocate volumes for our pods.

We will be defining each step to explain how we implemented this stack and use it to extract information from containers and use the beautiful Kibana dashboard.

5.3.2.1 EFK Architecture

The first step was to create a gitlab project with the EFK name so we can deploy our application into the internal cluster and use fluentd to extract containers from the other environments.

Fist when we create the project we added a kubernetes directory to host all the configuration and deployments as containers for the EFK stack, then we added a docker file to use logtrail plugin to kibana so we can view easier the container log and finally bind those info into a dynamic generated volumes with OpenEBS deployments.

The figure 5.14 below shows the project structure that we will explain later.

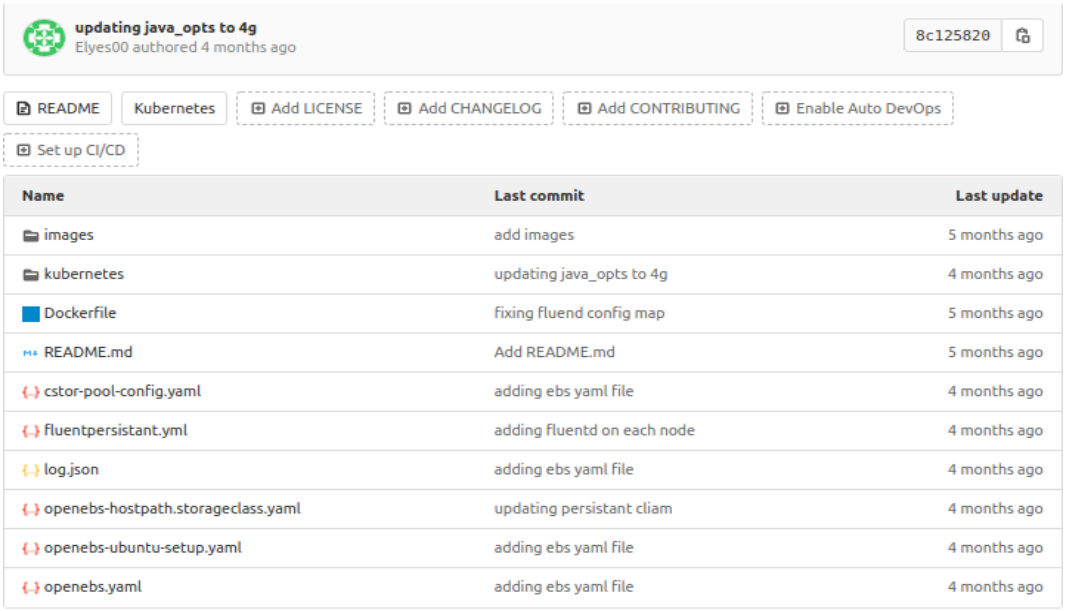


Figure 5.14: **EFK Gitlab Project Structure [37]**

5.3.2.2 EFK Deployments

This is where we will explian how we managed to deploy the EFK stack and how to link different components with each others but all the code and deployments files will be added in the annex section.

First of all we created our Docker file to add logtaril to the kibana container with this simple file shown in the 5.15 figure. In the second step we prepared the deployment file



Figure 5.15: **Logtrail Pluging [38]**

for the kibana image based on the image that we created with our Dockerfile adn use ConfigMap yaml file to add logtrail configuration inside the container, then we created the deployment, service, ServiceAccount, ClusterRole for rules injection of the elasticsearch. The only thing that remains after creating kibana and elasticsearch deployments is know-

ing how to bind them together to make those two services talk to each others.
Referring to ELK documentation we saw that those two components talk together through a defined 9200 port, so we added ”env” section into kibana deployment and added the elasticsearch URL so that kibana can reach the elasticsearch service.
When finishing with kibana and elasticsearch we created our fluentd DaemonSet to reach all the cluster and extract container logs.
With the same manner we added an ”env” section in the yaml file where we defined the host to which fluentd service must send the logs and in our case is elasticsearch so we used ”efk” namespace to refer to elasticsearch through ”FLUENTELASTICSEARCHHOST” = ”elasticsearch.efk”.
”elasticsearch.efk” values refer to the name of the deployment followed by the namespace where the elasticsearch was deployed in.
Finally we must add a persistent volume to our deployment so we can save all the logs and kibana dashboards created to track the cluster, so we used a PersistentVolumeClaim and a StorageClass to make it dynamically the allocation of volume through OpenEBS.
We take for example the fluentd pods, in fact fluentd reach each container and store the values of the logs in a persistent volume before send them to elasticsearch.First we define a VolumeMount in the container section where we can find the name and the path of the volume, secondly we add a Volume section to the pod so it can store the container data into a specific path related to the name of the volume created for the container and the name of the PersistentVolumeClaim name.
When we finish creating the pod specification we create our PersistentVolumeClaim yaml file where we specify the exact name added into the pod and the storageClassName with other field for storage and accessing rights. When this file is created we make our storageClass yaml file which refers to OpenEBS provision mechanism.

5.3.2.3 Container Tracking

To make an efficient tracking, we managed to create some beautiful dashboards so the Dev and Ops team can see easily what is going on with their containers and in which state they are with some logging information extracted with logtrail pluging.
As shown in those figures we have our graphs that shows containers per cluster (names , number and status) and container logs.

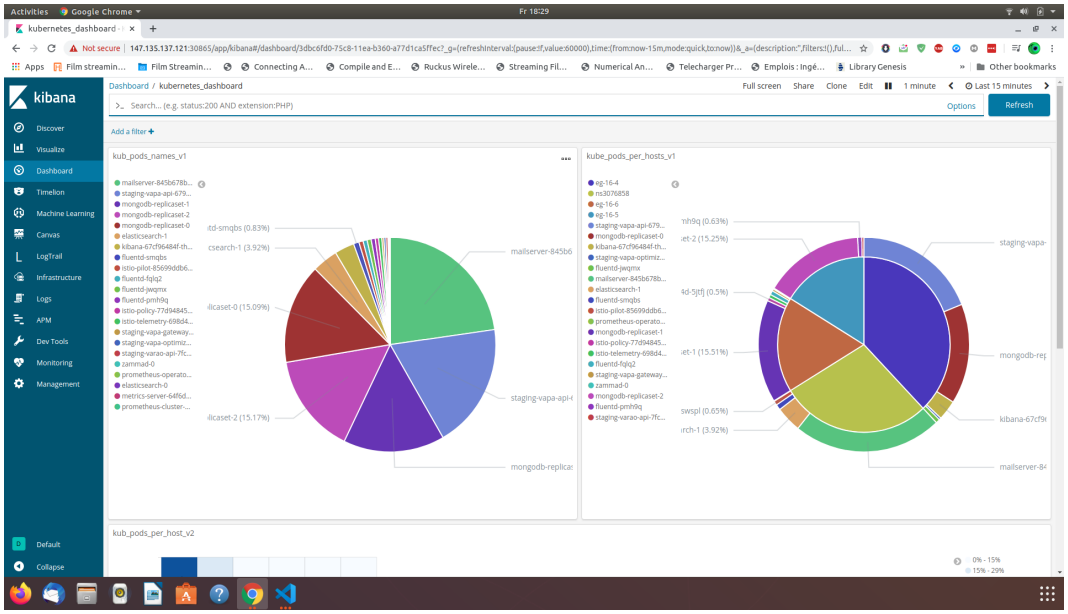


Figure 5.16: **Kibana Dashboard [39]**

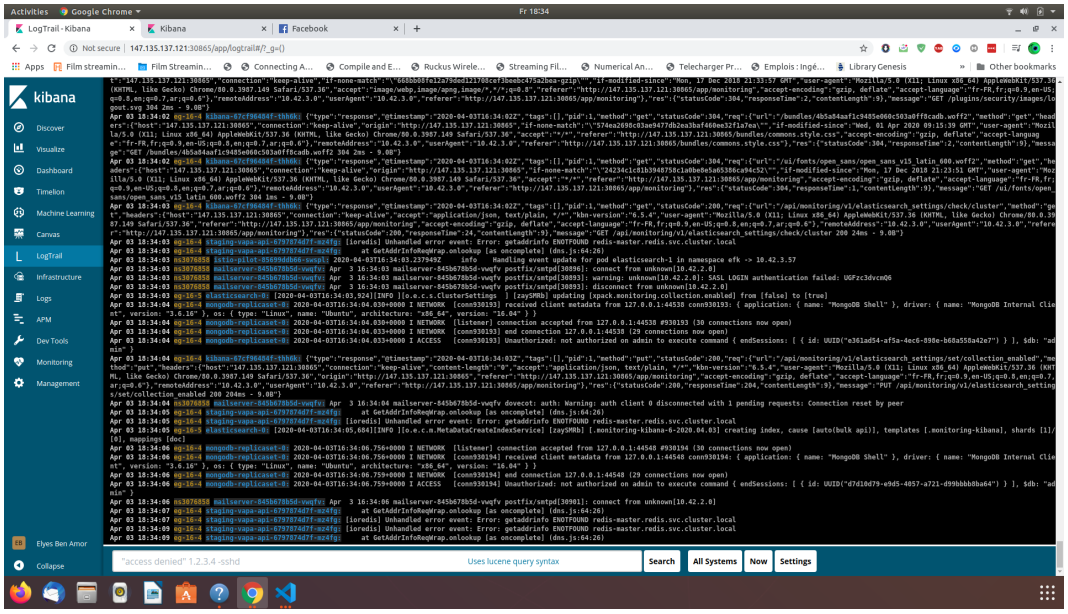


Figure 5.17: **Logtrail Dashboard [40]**

5.3.3 Prometheus Configuration

5.3.3.1 Integration

First we are going to install prometheus using prometheus operator project into our cluster using "kubectl" command, then we will deploy the prometheus adapter project and link it to the operator in order to manipulate its instance and orchestrate them with the proper resources. Finally using Bull project we will queue metrics from redis cache of our application, monotor it through a ServiceMonitor yaml file and then use those metrics for pod autoscaling.

As shown in figure 5.18, those are the components to install prometheus operator that we are going to expose and explain briefly. First of all we will deploy some rule based

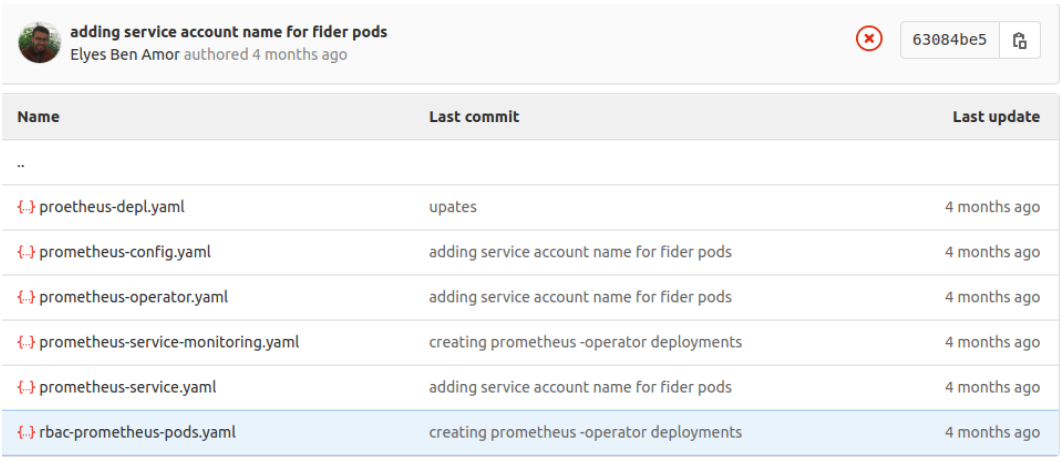


Figure 5.18: **Operator Project Structure**[41]

access control to our cluster in order to add some rule on how prometheus should access the cluster, secondly we will deploy the promethues instance all along with his service in a NodePort type so we can access prometheus application from a random cluster port and finally we will add a service monitor type to orchestrate prometheus instances as shown in figure 5.19.

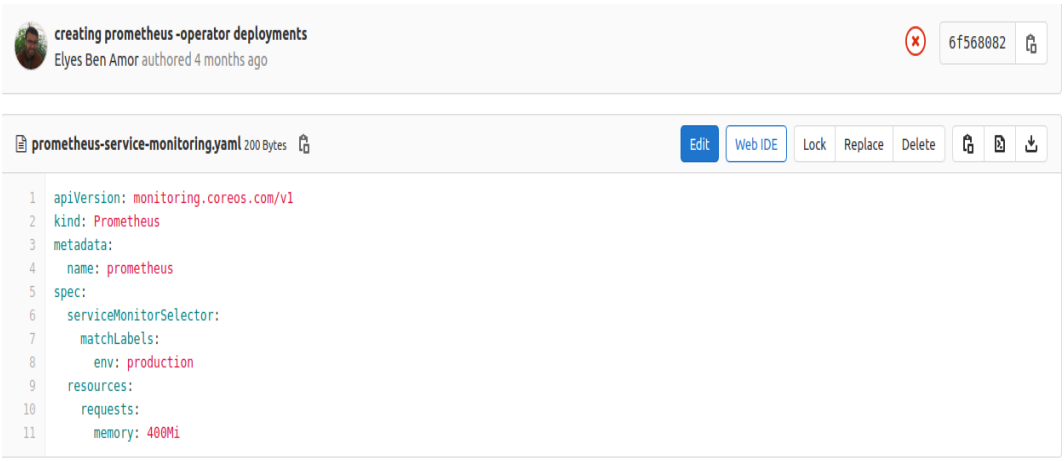


Figure 5.19: **Prometheus Service Monitore File[42]**

After installing prometheus we are going now to install prometheus adapter project through helm but first we must configure the adapter to listen to prometheus instances and add the external and custom metrics that has to scrape from the operator.
For that reason we chose to inspect the value.yaml file for helm chart of this project, adapt his values to our need and then install it with one line command.

1. **Inspect**: "helm inspect values stable/prometheus-adapter > adapter.values".This command will allow us to download values yaml file for the prometheus adapter project to adapt our needs.

2. **Modify File**: After having the file we are going to configure the base url of the prometheus instance as well as the rules as shown in those figures 5.20 and 5.21.

```
20 prometheus:
21   url: http://access-prometheus.cattle-prometheus.svc.cluster.local:8080/
22   port: 8080
23   path: ""
24
```

Figure 5.20: **URL Specification[43]**

```
rules:
  default: true
  custom:
  - seriesQuery: '{__name__=~"^some_metric_count$"}'
    resources:
      template: <<.Resource>>
    name:
      matches: ""
      as: "my_custom_metric"
    metricsQuery: sum(<<.Series>>{<<.LabelMatchers>>}) by (<<.GroupBy>>)
  - seriesQuery: '{namespace!="",__name__!~"^container_.*"}'
    seriesFilters:
    - isNot: .*_total$
    resources:
      template: <<.Resource>>
    name:
      matches: ""
      as: ""
    metricsQuery: sum(<<.Series>>{<<.LabelMatchers>>}) by (<<.GroupBy>>)
  - seriesQuery: '{namespace!="",__name__!~"^container_.*"}'
    seriesFilters:
    - isNot: .*_seconds_total
    resources:
      template: <<.Resource>>
    name:
      matches: ^(.*)_total$
      as: ""
    metricsQuery: sum(rate(<<.Series>>{<<.LabelMatchers>>}[1m])) by (<<.GroupBy>>)
  - seriesQuery: '{namespace!="",__name__!~"^container_.*"}'
    seriesFilters: []
    resources:
      template: <<.Resource>>
    name:
      matches: ^(.*)_seconds_total$
      as: ""
    metricsQuery: sum(rate(<<.Series>>{<<.LabelMatchers>>}[1m])) by (<<.GroupBy>>)


  # Mounts a configMap with pre-generated rules for use. Overrides the
  # default, custom, external and resource entries
  existing:
  external:
  - seriesQuery: '{__name__=~"^some_metric_count$"}'
    resources:
      template: <<.Resource>>
    name:
      matches: ""
      as: "my_external_metric"
    metricsQuery: sum(<<.Series>>{<<.LabelMatchers>>}) by (<<.GroupBy>>)
  - seriesQuery: 'http_requests_total{kubernetes_namespace!="",kubernetes_pod_name!=""}'
    resources:
      overrides:
        kubernetes_namespace: {resource: "namespace"}
        kubernetes_pod_name: {resource: "pod"}
```

Figure 5.21: **Rules Specification**[44]

3. **Installation**: After modifying the prometheus.values file we will install the project with this simple command: "$ helm install prometheus-adapter -f values.yaml stable/prometheus-adapter"

Now it is time to scrape our custom metrics from VARAO application using Bull project and as mentioned before to make it easier for prometheus to check new services deployed and configure automatically its config file, we are going to add a service monitor file for bull exporter so prometheus can autoscale its instance to discover this new service and srape metrics from bull configuration pods.
The figure 5.22 below show the bull exporter deployment with three environment necessary variable as referring to documentation:

- EXPORTER_AUTODISCOVER: To automatically discover metrics configured without using cron jobs.

- EXPORTER_REDIS_URL: Refers to redis url of the VARAO application from where to queue metrics.
- EXPORTER_QUEUES: Scrape the defined custom metrics defined in the application through endpoints.

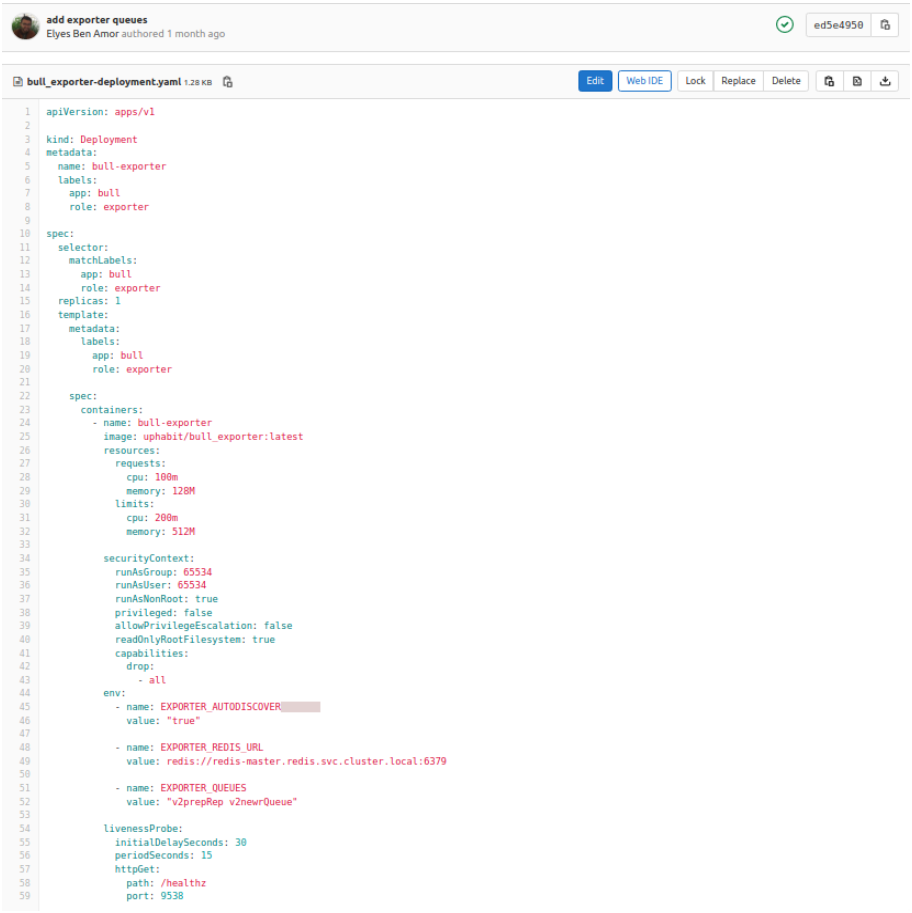


Figure 5.22: **Bull Deployment**[45]

After the deployment we must define its service that must have the selector type so it can refer to the deployment and also we have to add some prometheus annotation to tell prometheus that this service is ready to consume. Finally we will deploy this service as ClusterIP for internal communication as shown in figure 5.23.

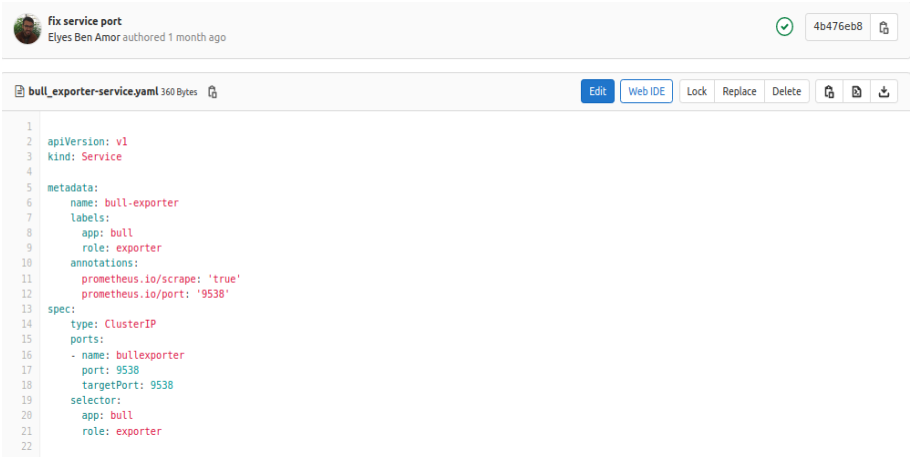


Figure 5.23: **Bull Service**[46]

And now came the last part of configuring this project by adding the service monitor concept as defined earlier by this figure 5.24.

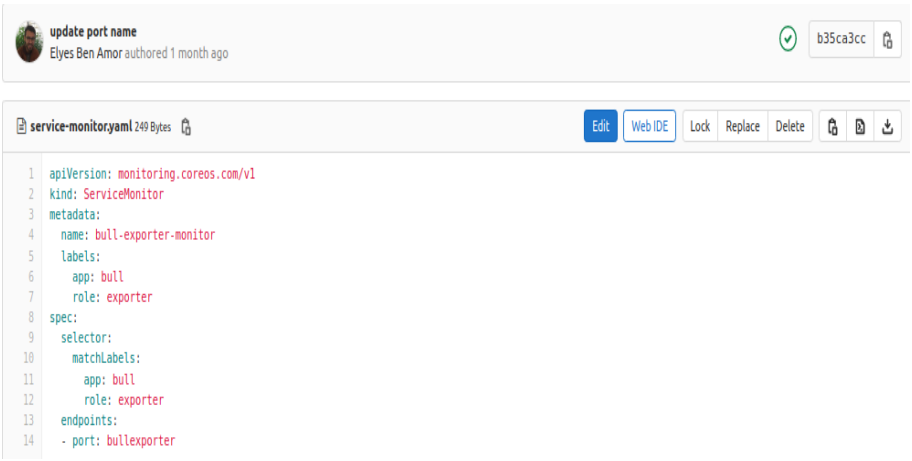


Figure 5.24: **Bull Service Monitor**[47]

5.3.3.2 Autoscaling

As mentioned before our application uses ingress service, so based on nginx we will scale our application through those requests of the metrics. In kubernetes it is simple to autoscale pods, you have just to define a HPA yalm file with a based configuration on what your application is consuming.

The figure 5.25 shows the HPA file where we define our desired replications based on custom objects.

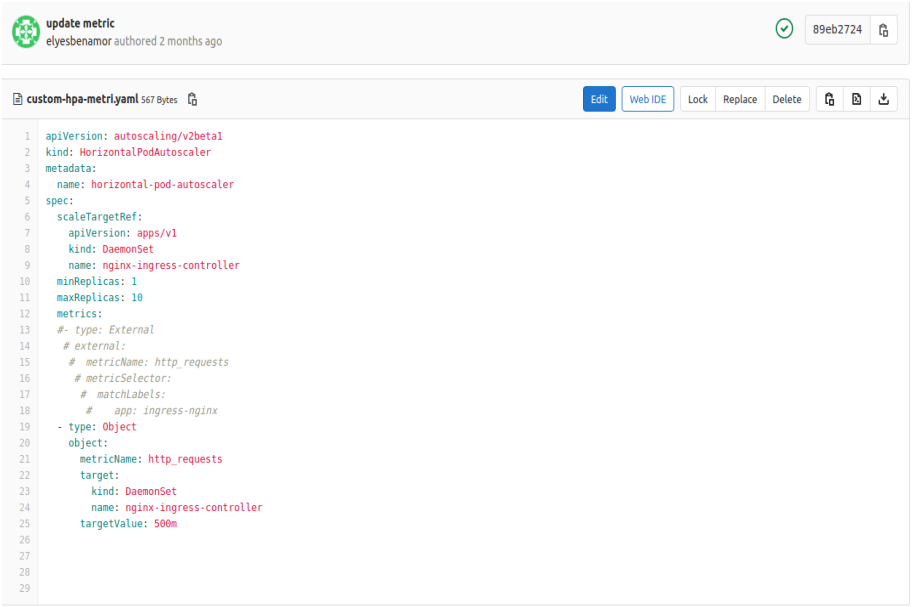


Figure 5.25: **HPA File**[48]

Conclusion

To conclude this chapter, we can say that we covered all the necessary pre-requisites to describe our internship goals. We also presented the steps to the release of the product and what we did to ensure efficiency.

The next chapter is a general conclusion of our work during the entire internship in the Vorwaerts company and we will dedicate also a part to give our perspective.

General conclusion and Perspectives

Globally, this internship was an important experience. Our work followed Vorwaerts GmBH's methodology throughout the product life cycle starting with a preliminary study where we introduced relevant concepts to our work and examined existing solutions to our system.

This has lead us to the design of our private cloud where we defined our cloud infrastructure components and presented several services by explaining each benefits and why containerizing application in production companies life. Then we detailed the CI/CD pipeline concept and the technology we used to implement it. Finally we presented the release of the product and how to monitor application with prometheus.

Our graduation projects consisted thus on migrating a SaaS application to private cloud and implement continuous solutions for adding more flexibility to our "dev" team to push their updates without fearing of conflicts. The graduation project didn't focus only on that but including also DevOps culture and practice in the whole process :

- Automating IT tasks like backup, testing, configuring to reduce errors.
- Combine development and operations teams to create a unilateral team that focuses on delivering common objectives.
- Improve productivity and reduce time to market.

Through this internship we saw the importance of the DevOps life cycle with the SCRUM methodology to add more values to product release and to make it worth it in how operating while automating everything.

Bibliography

[1] Vorwaerts:

https://fr.wikipedia.org/wiki/Vorwärts

[visited on : 20/08/2020]

[2] VAPA:

https://www.vorwaerts.com/vapa/en/vapa

[visited on : 20/08/2020]

[3] VARAO:

https://www.vorwaerts.com/varao/en/

[visited on : 20/08/2020]

[4] CDP:

https://tool.clubderprodukttester.de/

[visited on : 20/08/2020]

[5] SCRUM:

https://www.researchgate.net/publication/268334807SoftwareDevelopmentLifeCycleAGII

[visited on : 22/08/2020]

[6] SCRUM:

https://www.atlassian.com/agile/scrum

[visited on : 22/08/2020]

[7] DEVOPS:

https://www.edureka.co/blog/devops-lifecycle/

[visited on : 23/08/2020]

[8] DEVOPS, Lucy Ellen Lwakatare:
`DevOps Adoption And Implementation In Software Development Practice`.
Released on 27 November 2017.
[Visited on : 23/08/2020]

[9] DEVOPS:
`https://www.sourceallies.com/2015/12/lessons-from-devops-experience`
[visited on : 23/08/2020]

[10] AMAZON:
`https://aws.amazon.com/fr/`
[visited on : 25/08/2020]

[11] Michael Collier Robin Shahan:
`Microsoft Azure Essentials, Second Edition`.
[Visited on : 25/08/2020]]

[12] HEROKU:
`https://www.heroku.com/platform`
[visited on : 25/08/2020]

[13] IBM:
`https://cloud.ibm.com/docs`
[visited on : 25/08/2020]

[14] GCP:
`https://cloud.google.com/why-google-cloud`
[visited on : 26/08/2020]

[15] OVH:
`https://www.ovhcloud.com/fr/about-us/`
[visited on : 26/08/2020]

[16] AMAZON S3:
`https://aws.amazon.com/s3/?nc1=h_ls`
[visited on : 26/08/2020]

[17] DOCKER:
https://www.zdnet.com/article/what-is-docker-and-why-is-it-so-darn-popular/
[visited on : 26/08/2020]

[18] KUBERNETES:
https://www.educative.io/courses/practical-guide-to-kubernetes/NEx4zlXE4P2
[visited on : 26/08/2020]

[19] KUBERNETES:
https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/
[visited on : 26/08/2020]

[20] KUBERNETES:
https://kubernetes.io/docs/concepts/overview/components/
[visited on : 26/08/2020]

[21] KUBERNETES:
https://medium.com/jorgeacetozi/kubernetes-master-components-etcd-api-server-controller-manager-and-scheduler-3a0179fc8186
[visited on : 27/08/2020]

[22] RANCHER:
https://rancher.com/docs/rancher/v2.x/en/overview/
[visited on : 27/08/2020]

[23] KANIKO:
https://medium.com/01001101/enhance-your-docker-image-build-pipeline-with-kaniko-567afb6cf97c
[visited on : 27/08/2020]

[24] HELM:
https://www.digitalocean.com/community/tutorials/an-introduction-to-helm-the-package-manager-for-kubernetes
[visited on : 30/08/2020]

[25] OPENEBS:
`https://docs.openebs.io/docs/next/overview.html`
[visited on : 30/08/2020]

[26] PROMETHEUS:
`https://prometheus.io/docs/introduction/overview/`
[visited on : 20/08/2020]

[27] PROMETHEUS:
`https://devops.college/prometheus-operator-how-to-monitor-an-external-service-3cb6ac8d5acb`
[visited on : 01/09/2020]

[28] PROMETHEUS:
`https://github.com/DirectXMan12/k8s-prometheus-adapter`
[visited on : 03/09/2020]

[29] EFK:
`https://www.digitalocean.com/community/tutorials/how-to-set-up-an-elasticsearch-fluentd-and-kibana-efk-logging-stack-on-kubernetes`
[visited on : 03/09/2020]

[30] BULL:
`https://github.com/OptimalBits/bull`
[visited on : 05/09/2020]

[31] HPA:
`https://blog.kloia.com/kubernetes-hpa-externalmetrics-prometheus-acb1d8a4ed50`
[visited on : 06/09/2020]

[32] HPA:
`https://thenewstack.io/scale-applications-in-kubernetes-with-kubectl-and-the-horizontal-pod-autoscaler/`
[visited on : 06/09/2020]